SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

백트래킹

Python

학습시작

미로를 빠져나오는 법을 컴퓨터가 스스로 알아내게 할 수 있을까요?

1. 여러 경로를 탐색하다가 조건에 맞지 않으면 다시 돌아가는 방식을 [ ]이라고 합니다.
2. 탐색 중에 가능성이 없는 경우를 미리 제외하는 과정을 [ ]라고 합니다.

- 가능한 경로를 모두 시도하고 실패 시 되돌아 갈 수 있습니다.
- 불필요한 시도는 생략하는 전략을 익히게 됩니다.

Objectives

학습목표

- 백트래킹 개념을 학습하고 최적화 문제와 결정 문제에 적용할 수 있음을 이해한다.
- 백트래킹으로 상태 공간 트리를 탐색하는 방법에 대해 이해한다.
- 이진 트리의 순회에 대해 설명할 수 있다.
- 이진탐색트리의 연산에 대해 설명할 수 있다.
- 힙 트리에 대해 설명할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

백트래킹

- 백트래킹 응용
- 연습 문제

트리

- 트리 개요
- 이진 트리
- 연습 문제
- 이진탐색트리
- 힙

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

백트래킹

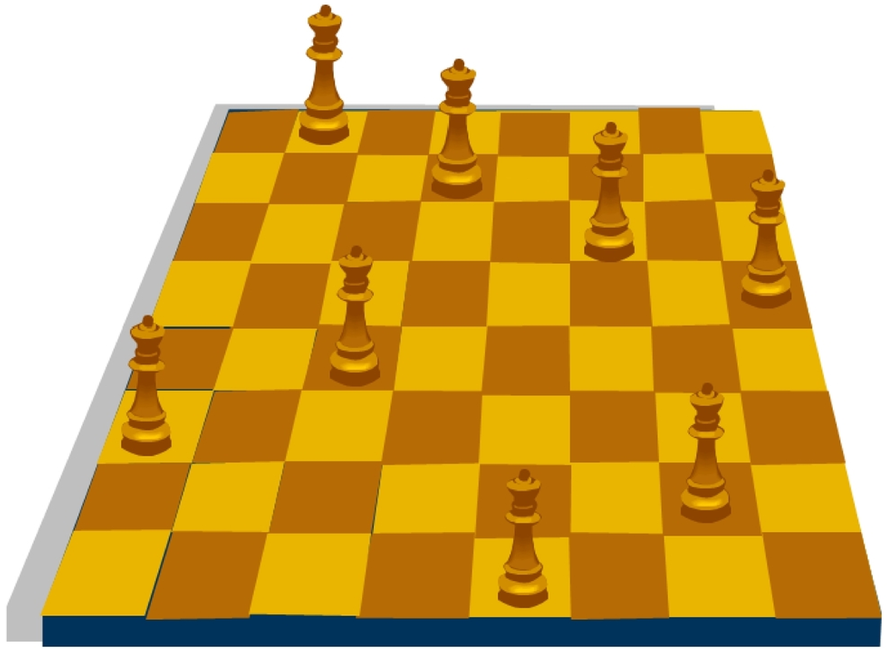
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

백트래킹 응용

문제 제시 : N-Queen 문제

- n*n 서양 장기판에서 배치한 Queen들이 서로 위협하지 않도록 n개의 Queen을 배치하는 문제
  - 어떤 두 Queen도 서로를 위협하지 않아야 한다.
  - Queen을 배치한 n개의 위치는?



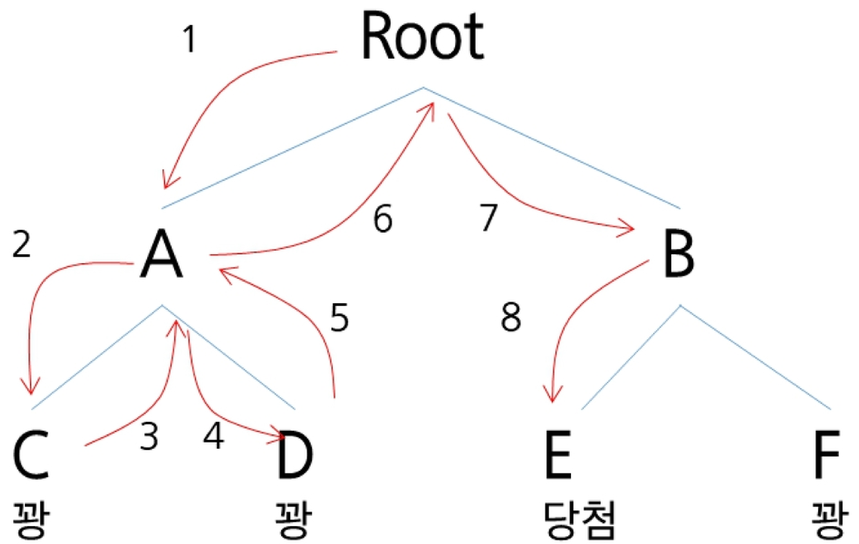
<그림1_ SW문제해결 응용 _백트래킹_N-Queen 문제>

백트래킹(Backtracking) 개념

- 여러 가지 선택지(옵션)들이 존재하는 상황에서 한 가지를 선택함
- 선택이 이루어지면 새로운 선택지들의 집합이 생성됨
- 이런 선택을 반복하면서 최종 상태에 도달함
  - 올바른 선택을 계속하면 목표 상태(goal state)에 도달합니다.

당첨 리프 노드 찾기

- 루트에서 갈 수 있는 노드를 선택함
- 꽝 노드까지 도달하면 최근의 선택으로 되돌아와서 다시 시작
- 더 이상의 선택지가 없다면 이전의 선택지로 돌아가서 다른 선택함
- 루트까지 돌아갔을 경우 더 이상 선택지가 없다면 찾는 답이 없음



<그림2_ SW문제해결 응용 _백트래킹_리프 노드 찾기>

백트래킹과 깊이 우선 탐색과의 차이

- 어떤 노드에서 출발하는 경로가 해결책으로 이어질 것 같지 않으면 더 이상 그 경로를 따라가지 않음으로써 시도의 횟수를 줄임

  - 이를 Pruning(가지치기)라고 합니다.

- 깊이 우선 탐색이 모든 경로를 추적하는데 비해 백트래킹은 불필요한 경로를 조기에 차단
- 깊이 우선 탐색을 가하기에는 경우의 수가 너무나 많은 경우, 즉, N! 가지의 경우의 수를 가진 문제에 대해 깊이 우선 탐색을 가하면 당연히 처리 불가능한 문제가 됨
- 백트래킹 알고리즘을 적용하면 일반적으로 경우의 수가 줄어들지만, 이 역시 최악의 경우에는 여전히 지수 함수 시간 (Exponential Time)을 요하므로 처리 불가능함

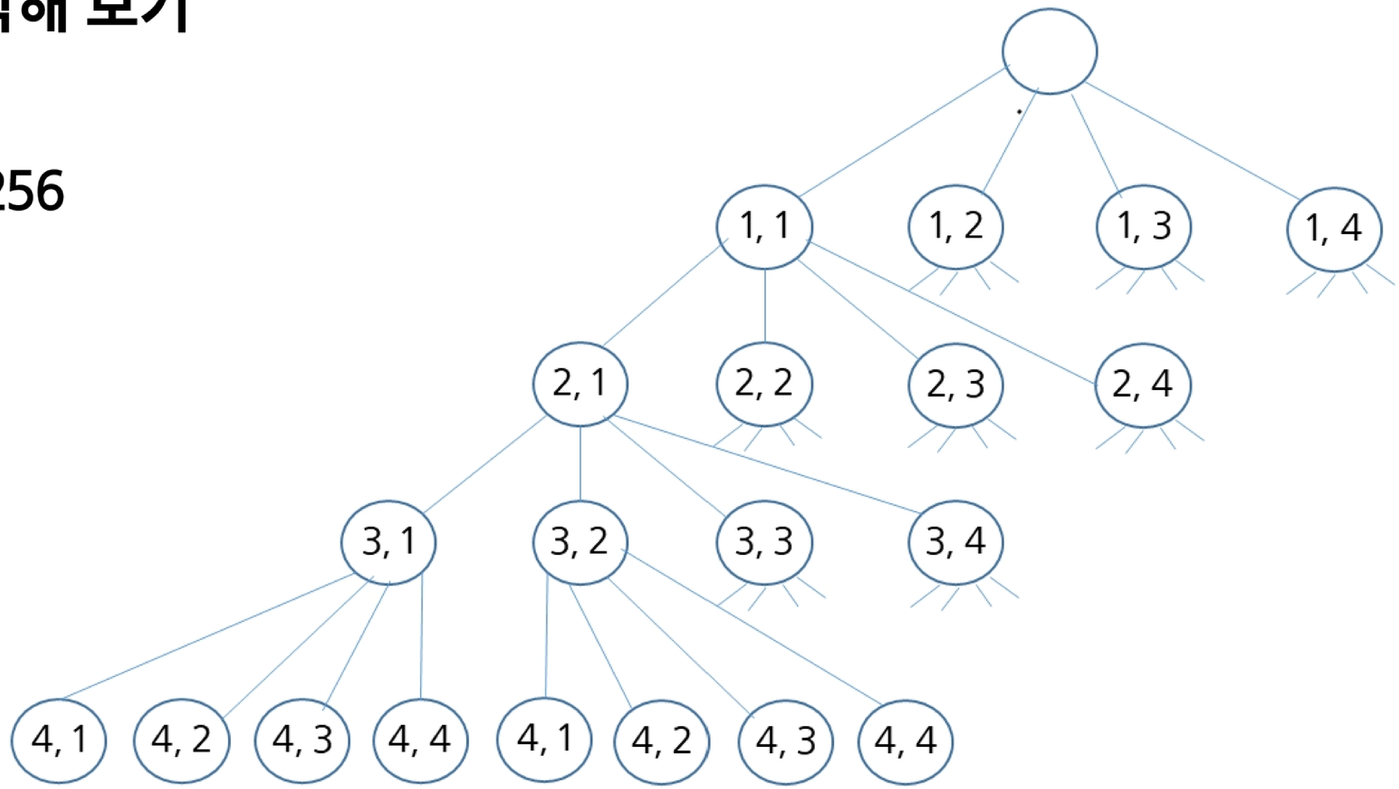
8-Queens 문제

- 퀸 8개를 8x8 크기의 체스판 안에 서로를 공격할 수 없도록 배치하는 모든 경우를 구하는 문제
- 후보 해의 수 : $${}_{64}C_8 = \frac{64!}{8!\,(64-8)!} = 4{,}426{,}165{,}368$$
- 실제 해의 수 : 이 중에서 실제 해는 92개 뿐
- 즉, 44억 개가 넘는 후보 해의 수 속에서 92개를 최대한 효율적으로 찾아내는 것이 관건

4-Queens 문제로 축소해서 생각해 보기

- 같은 행에 위치할 수 없음
- 모든 경우의 수 : 4×4×4×4 = 256

- 코드 : 1.N-queen.py



상태 공간 트리(state space tree)

<그림3_ SW문제해결 응용 _백트래킹_상태 공간 트리>

백트래킹 개념

- 루트 노드에서 리프(leaf) 노드까지의 경로는 해답후보(candidate solution)가 되는데, 깊이 우선 검색을 하여 그 해답후보 중에서 해답을 찾을 수 있음
- 그러나 이 방법을 사용하면 해답이 될 가능성이 전혀 없는 노드의 후손 노드(descendant)들도 모두 검색해야 하므로 비효율적

※ 그렇다면 모든 후보를 검사하는가? No!

백트래킹 기법

- 어떤 노드의 유망성을 점검한 후에 유망(promising)하지 않다고 결정되면 그 노드의 부모로 되돌아가(backtracking) 다음 자식 노드로 감
- 어떤 노드를 방문하였을 때 그 노드를 포함한 경로가 해답이 될 수 없으면 그 노드는 유망하지 않다고 하며, 반대로 해답의 가능성이 있으면 유망하다고 함
- 가지치기(pruning) : 유망하지 않는 노드가 포함되는 경로는 더 이상 고려하지 않음

백트래킹을 이용한 알고리즘의 절차

① 상태 공간 트리의 깊이 우선 검색을 실시한다.

② 각 노드가 유망한지를 점검한다.

③ 만일 그 노드가 유망하지 않으면, 그 노드의 부모 노드로 돌아가서 검색을 계속한다.

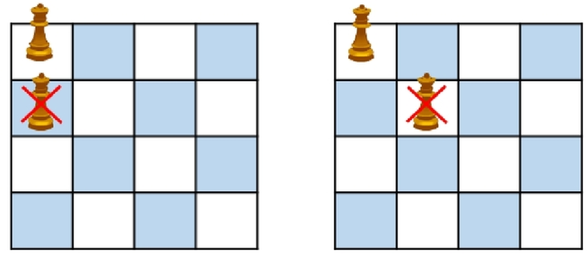
N-Queen 문제

- N개의 퀸을 놓을 수 있는 경우의 수

- 코드 : 1.N-queen.py

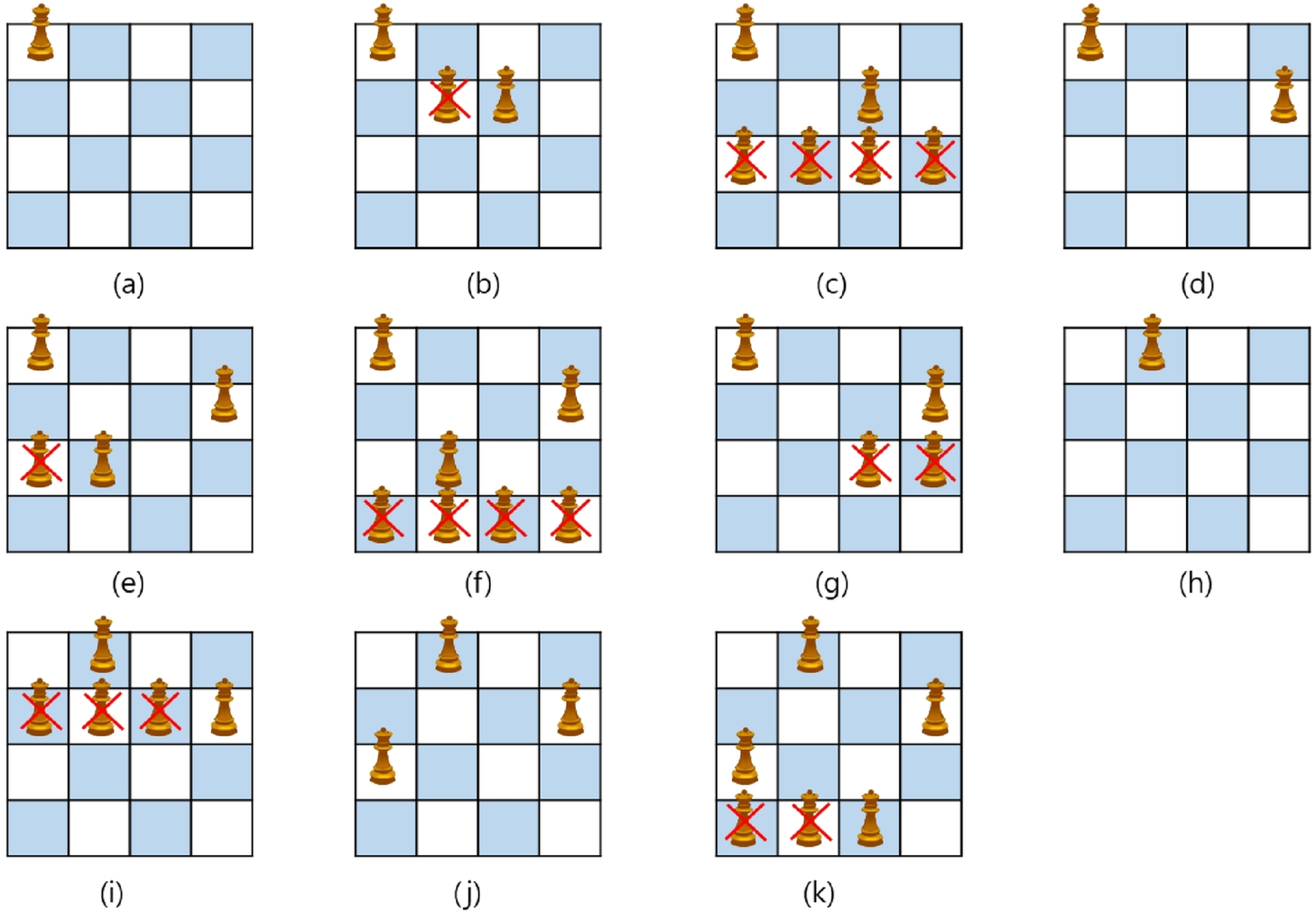
```
def recur(row):
    global answer

    if row == N:
        answer += 1
        return
    for col in range(N):
        # 가지치기 : 같은 열을 못 고르도록
        #  --> 유망하지 않은 케이스를 모두 삭제 (세로, 대각선)
        if check(row, col) is False:
            continue
        # col을 선택했다
        visited[row][col] = 1
        recur(row + 1)
        visited[row][col] = 0
```



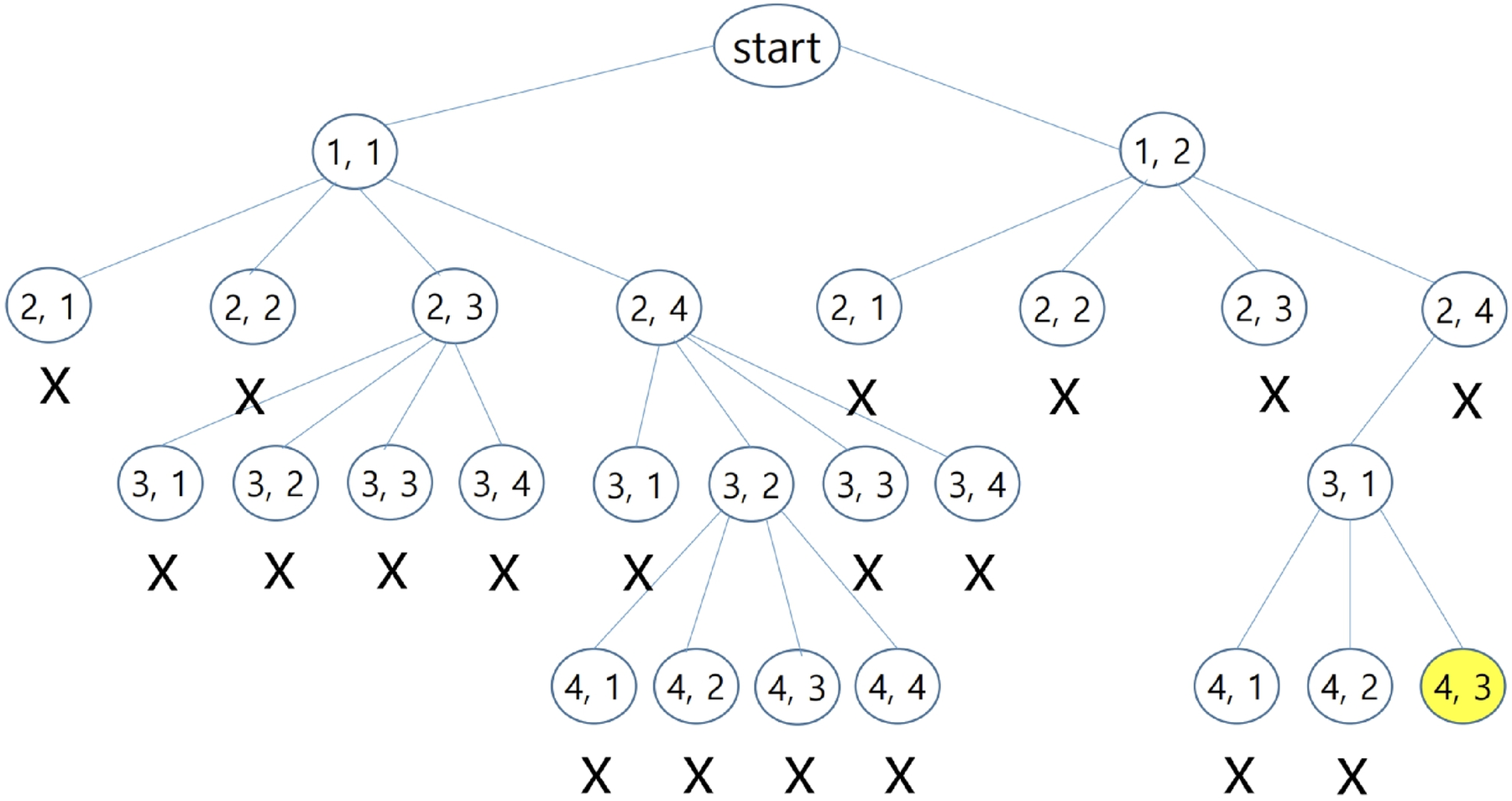
<그림4_ SW문제해결 응용 _백트래킹_일반 백트래킹 알고리즘>

일반 백트래킹 알고리즘



<그림5_ SW문제해결 응용 _백트래킹_일반 백트래킹 알고리즘의 흐름도>

상태 공간 트리



<그림6_ SW문제해결 응용 _백트래킹_상태 공간 트리의 예시>

깊이 우선 탐색과 백트래킹 비교

- 최초의 해를 찾기까지의 노드 방문 횟수
  - 순수한 깊이 우선 검색 = 155 노드
  - 백트래킹 = 27 노드

상태 공간 트리를 구축하여 문제를 해결 (해의 존재 유무 화인)

```
bool backtrack(선택 집합, 선택한 수, 모든 선택 수)
{
        if (선택한 수 == 모든 선택 수) // 더 이상 탐색할 노드가 없다.
        {
                찾는 솔루션인지 체크;
                return 결과;
        }
        현재 선택한 상태집합에 포함되지 않는 후보 선택들(노드) 생성

        모든 후보 선택들에 대해
        {
                선택 집합에 하나의 후보선택을 추가
                선택한 수 = 선택한 수 + 1
                결과 = backtrack 호출(선택집합, 선택한 수, 모든 선택수)

                if (결과 == 성공)
                        return 성공; // 성공한 경우 상위로 전달
        }
        return 실패;
}
```

연습 문제

{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}의 powerset 중 원소의 합이 10인 부분집합을 모두 출력하시오.

- 코드 : 2.powerset_problem.py

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

트리

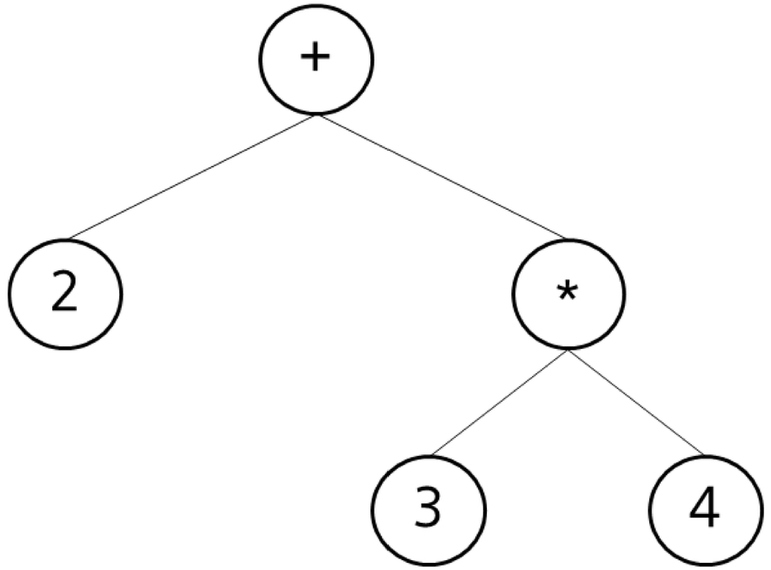
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

트리 개요

문제 제시 : 계산기

- 수식 2 + 3 * 4 를 다음과 같은 그래프로 표현하고 그래프를 순회하여 수식을 계산하는 문제

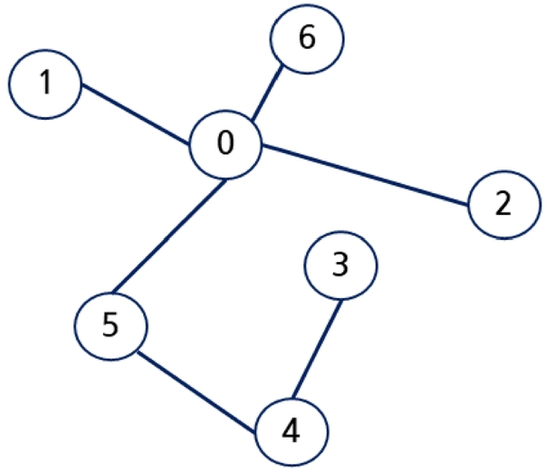


<그림7_ SW문제해결 응용 _백트래킹_계산기>

트리(Tree)

- 트리는 싸이클이 없는 무향 연결 그래프임
  - 두 노드(or 정점) 사이에는 유일한 경로가 존재합니다.
  - 각 노드는 최대 하나의 부모 노드가 존재할 수 있습니다.
  - 각 노드는 자식 노드가 없거나 하나 이상이 존재할 수 있습니다.

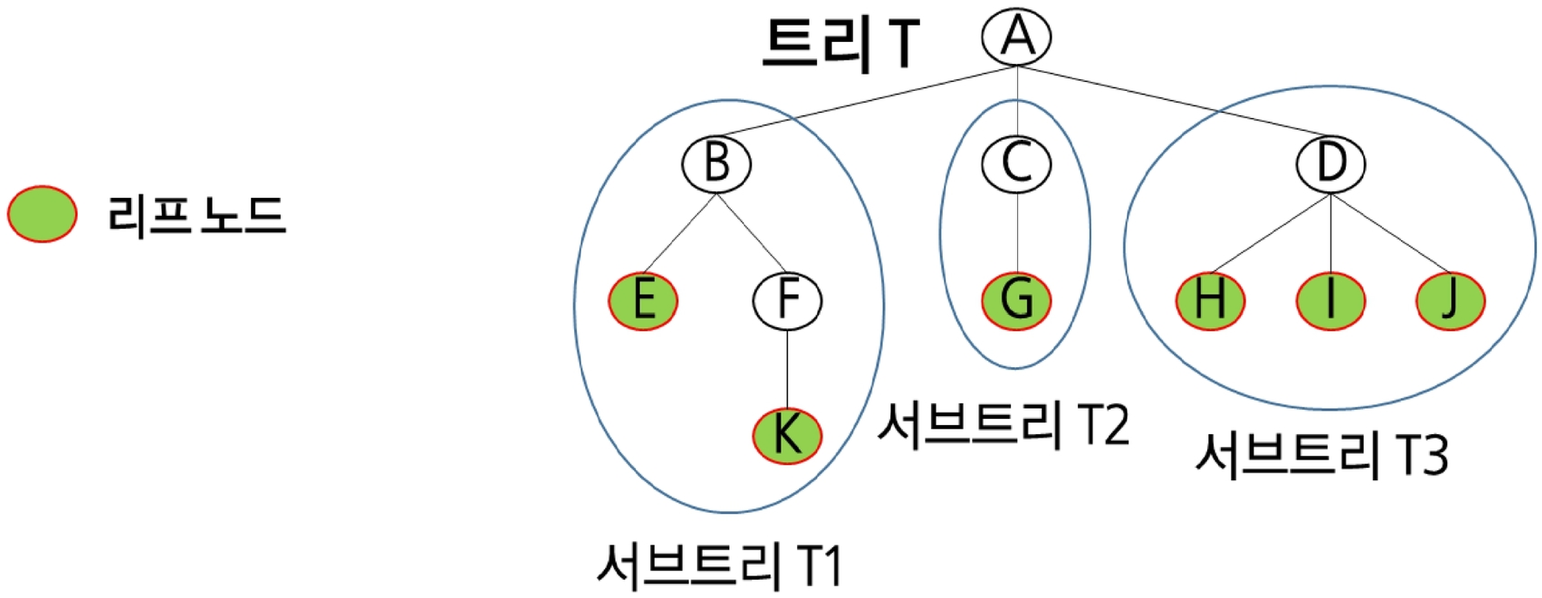
- 비선형 구조
  - 원소들 간에 1:n 관계를 가지는 자료구조 입니다.
  - 원소들 간에 계층관계를 가지는 계층형 자료구조 입니다.



<그림8_ SW문제해결 응용 _백트래킹_ 트리>

트리의 정의

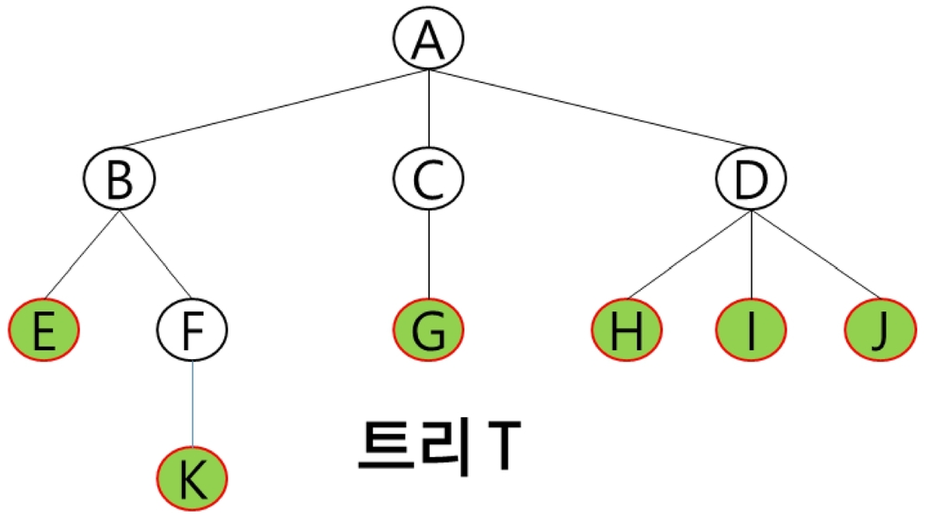
- 한 개 이상의 노드로 이루어진 유한 집합이며 다음 조건을 만족함
  - 노드 중 최상위 노드를 루트(root)라고 합니다.
  - 나머지 노드들은 n(>=0)개의 분리 집합 T1, …, TN으로 분리될 수 있습니다.
- 이들 T1, …, TN은 각각 하나의 트리가 되며(재귀적 정의) 루트의 부분 트리(subtree)라 함



<그림9_ SW문제해결 응용 _백트래킹_ 트리의 정의>

트리 용어

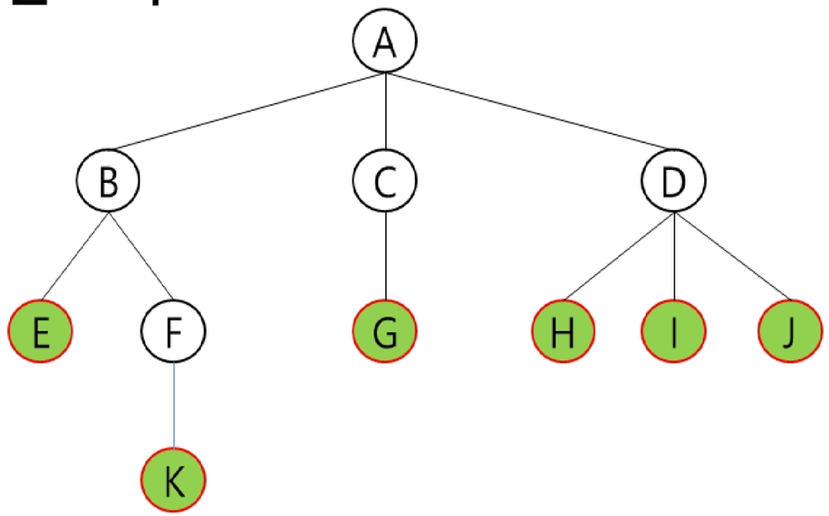
- 노드 (node) : 트리의 원소이고 정점 (vertex)이라고도 함
  - 트리 T의 노드 - A, B, C, D, E, F, G, H, I, J, K
- 간선 (edge) : 노드를 연결하는 선.
  - 부모 노드와 자식 노드를 연결합니다.
- 루트 노드 (root node) : 트리의 시작 노드
  - 트리 T의 루트 노드는 A 입니다.
- 리프 노드(leaf node) : 자식 노드가 없는 노드
  - E, K, G, H, I, J는 리프 노드 입니다.



<그림10_ SW문제해결 응용 _백트래킹_ 트리의 구성요소>

트리 용어

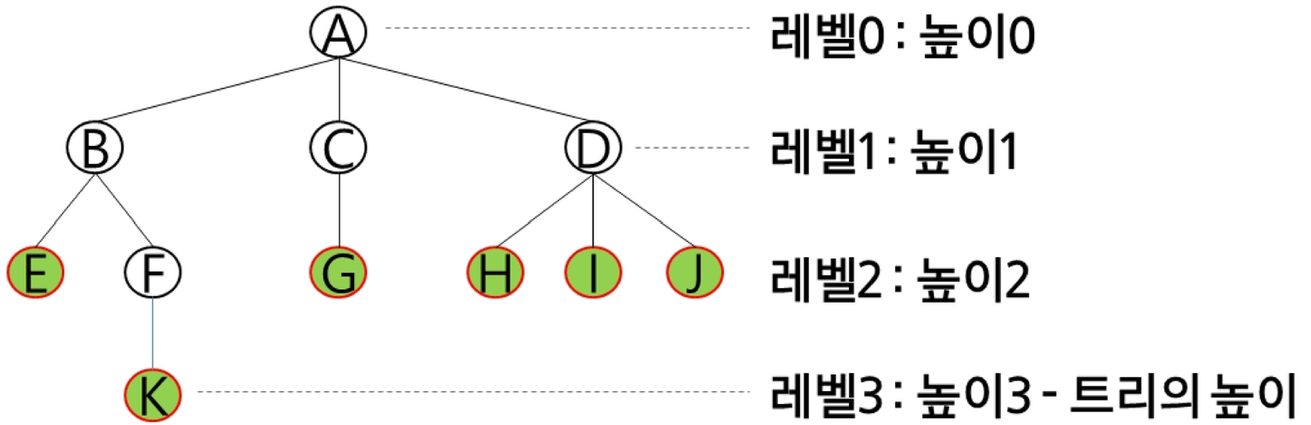
- 형제 노드 (sibling node) - 같은 부모 노드의 자식 노드들
  - B, C, D 는 형제 노드 입니다.
- 조상 노드 - 간선을 따라 루트 노드까지 이르는 경로에 있는 모든 노드들
  - K의 조상 노드는 F, B, A 입니다.
- 서브 트리 (subtree) - 부모 노드와 연결된 간선을 끊었을 때 생성되는 트리
- 자손 노드 - 서브 트리에 있는 하위 레벨의 노드들
  - B의 자손 노드는 E, F, K 입니다.



<그림11_ SW문제해결 응용 _백트래킹_트리의 용어>

트리 용어

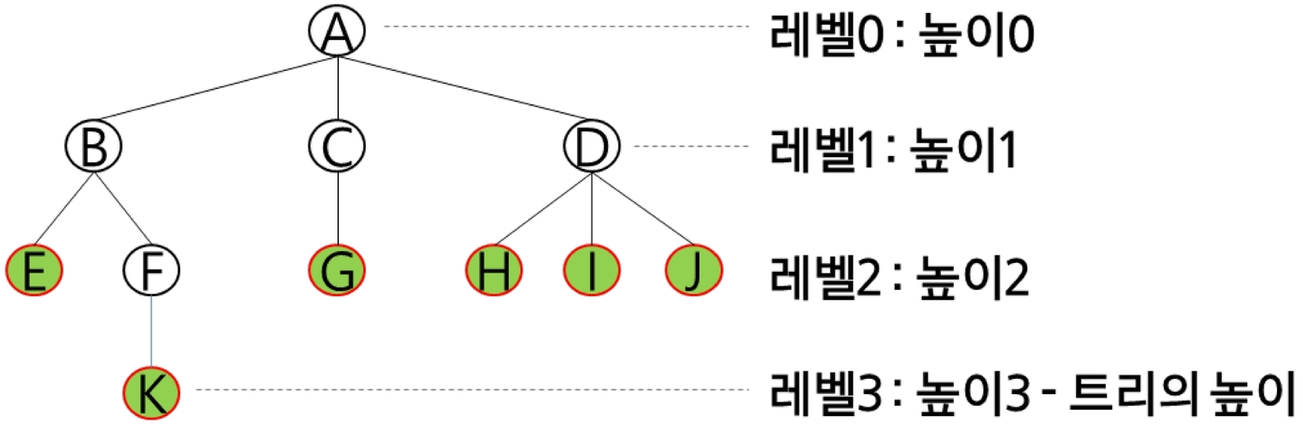
- 노드의 차수 : 노드에 연결된 자식 노드의 수
  - B의 차수 = 2, C의 차수 = 1
- 트리의 차수 : 트리에 있는 노드의 차수 중에서 가장 큰 값
  - 트리 T의 차수 = 3
- 단말 노드 (리프 노드) : 차수가 0인 노드. 자식 노드가 없는 노드



<그림12_ SW문제해결 응용 _백트래킹_ 트리의 차수>

트리 용어

- 노드의 높이 - 루트에서 노드에 이르는 간선의 수. 노드의 레벨
  - B의 높이 = 1, F의 높이 = 2
- 트리의 높이 - 트리에 있는 노드의 높이 중에서 가장 큰 값. 최대 레벨
  - 트리 T의 높이 = 3



<그림13_ SW문제해결 응용 _ 백트래킹_ 트리의 높이>

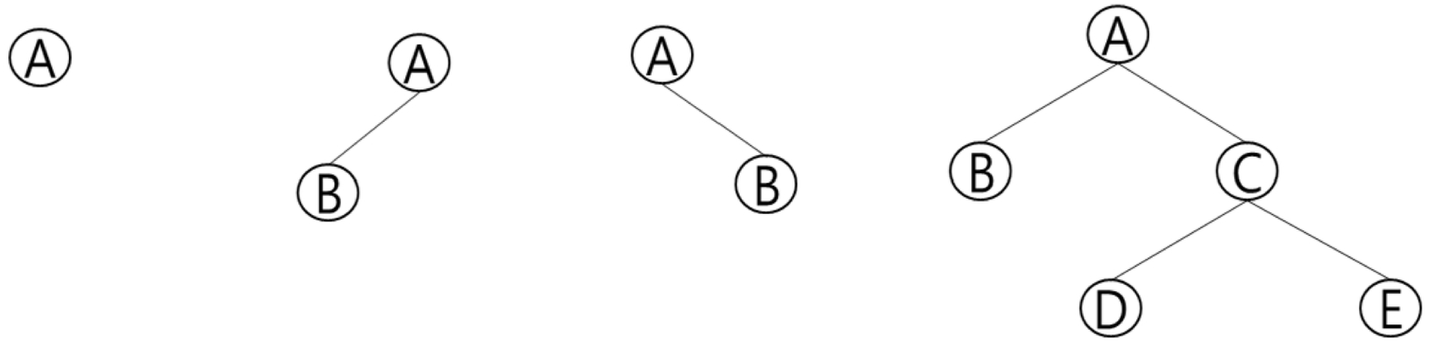
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 트리

이진 트리 (Binary Tree)

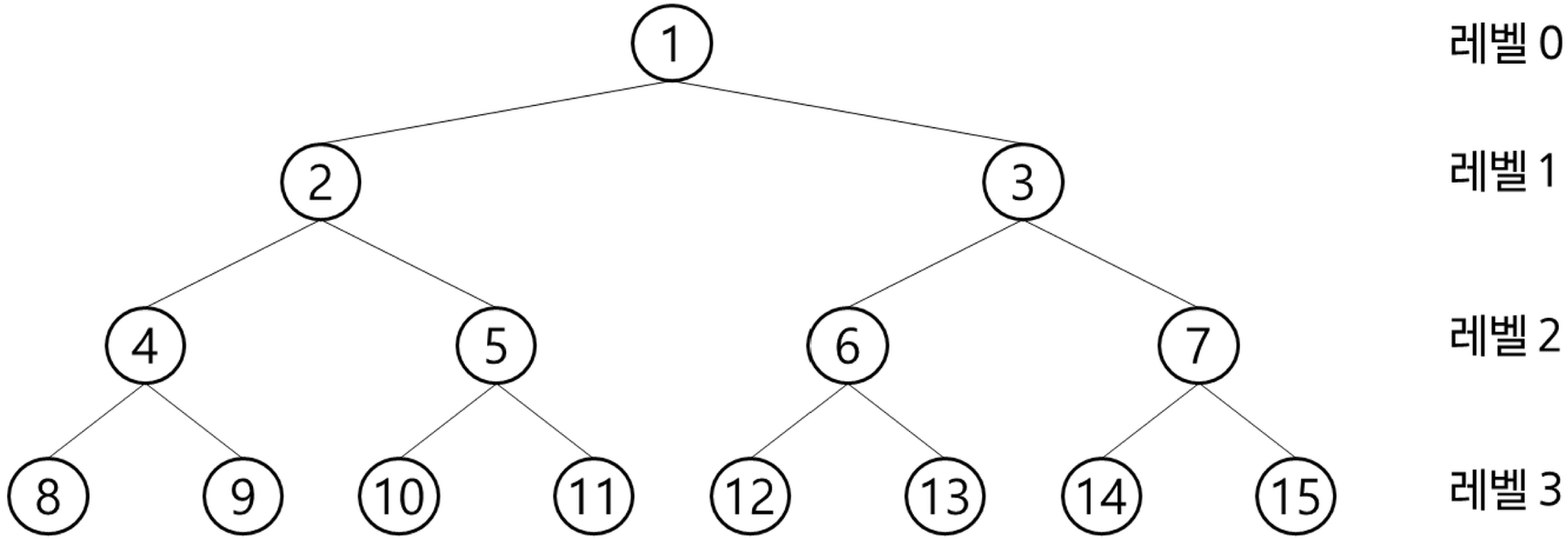
- 모든 노드들이 최대 2개의 서브트리를 갖는 특별한 형태의 트리
- 각 노드가 자식 노드를 최대한 2개 까지만 가질 수 있는 트리
  - 왼쪽 자식 노드 (left child node)
  - 오른쪽 자식 노드 (right child node)
- 이진 트리의 예



<그림14_ SW문제해결 응용 _ 백트래킹_ 이진 트리의 예>

이진 트리의 특성

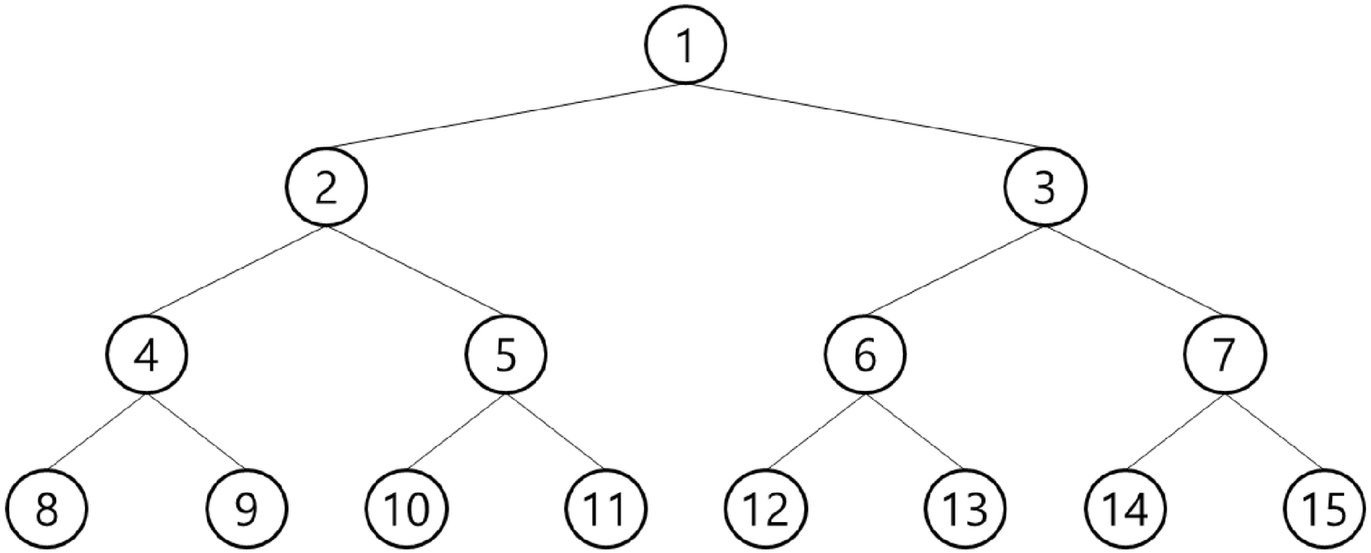
- 레벨 i에서의 노드의 최대 개수는 2^i개
- 높이가 h인 이진 트리가 가질 수 있는 노드의 최소 개수는 (h+1)개가 되며, 최대 개수는 $(2^{h+1}-1)$개



<그림15_ SW문제해결 응용 _ 백트래킹_ 이진 트리의 특성>

포화 이진 트리 (Full Binary Tree)

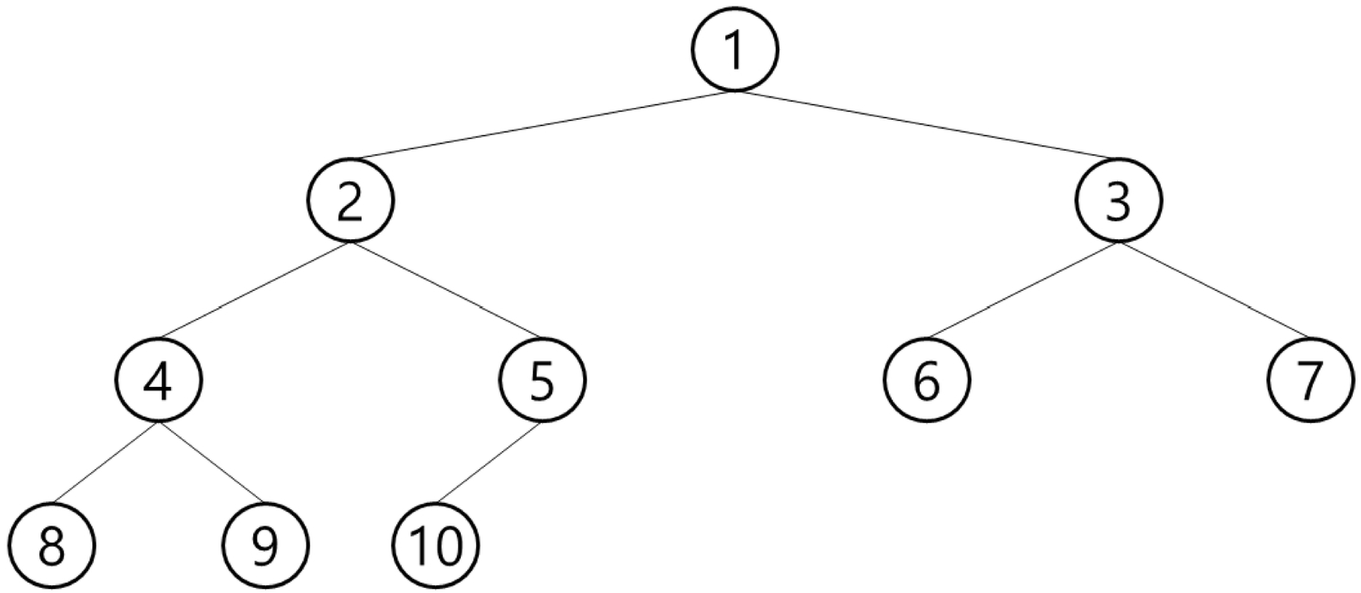
- 모든 레벨에 노드가 포화상태로 차 있는 이진 트리
- 높이가 h일 때, 최대의 노드의 개수인 $(2^{h+1}-1)$의 노드를 가진 이진 트리
  - 높이 3일 때 $2^{3+1}-1=15$개의 노드
- 루트를 1번으로 하여 $2^{h+1}-1$까지 정해진 위치에 대한 노드 번호를 가짐



<그림16_ SW문제해결 응용 _ 백트래킹_ 포화 이진 트리>

완전 이진 트리 (Complete Binary Tree)

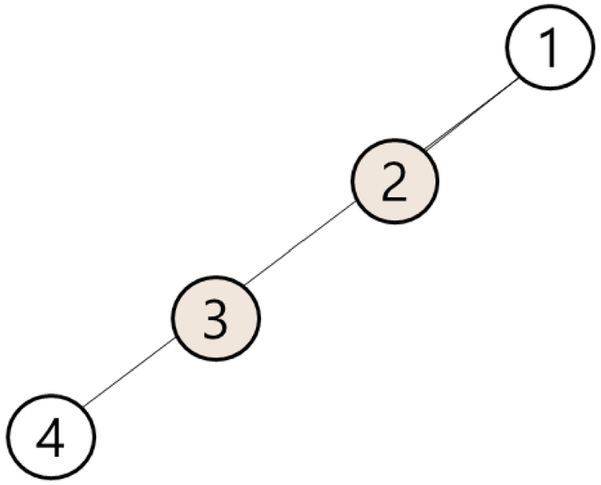
- 높이가 h이고 노드 수가 n개일 때 (단, $2^h \le n \le 2^{h+1}-1$), 포화 이진 트리의 노드 번호 1번부터 n번까지 빈 자리가 없는 이진 트리
  - 예) 노드가 10개인 완전 이진 트리



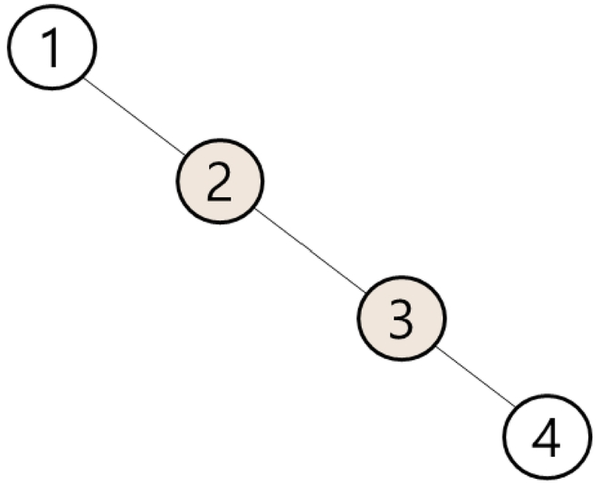
<그림17_ SW문제해결 응용 _ 백트래킹_ 완전 이진 트리>

편향 이진 트리 (Skewed Binary Tree)

- 높이가 h에 대한 최소 개수의 노드를 가지면서 한쪽 방향의 자식 노드 만을 가진 이진 트리
  - 왼쪽 편향 이진 트리
  - 오른쪽 편향 이진 트리



<그림18_ SW문제해결 응용 _ 백트래킹_ 왼쪽 편향 이진 트리>



<그림19_ SW문제해결 응용 _ 백트래킹_ 오른쪽 편향 이진 트리>

순회

순회

트리의 각 노드를 중복되지 않게 전부 방문(visit)하는 것

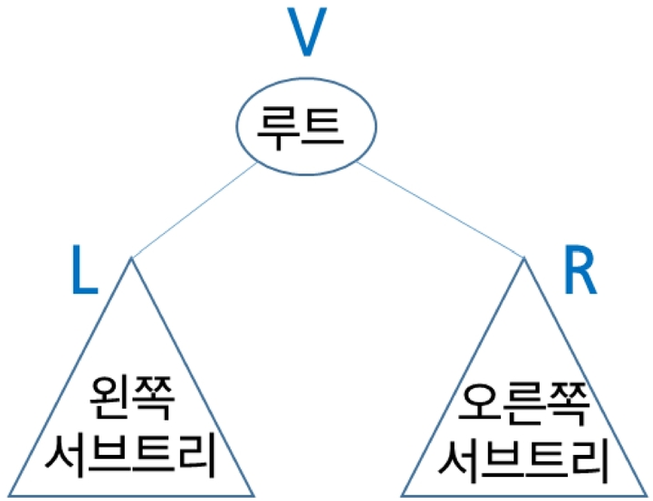
트리는 비선형 구조이기 때문에 선형 구조에서와 같이
선후 연결 관계를 알 수 없습니다.

순회 (traversal)

- 트리의 노드들을 체계적으로 방문하는 것

3가지의 기본적인 순회 방법

- 전위순회 (preorder traversal) : VLR
  - 자손 노드보다 현재 노드를 먼저 방문한다.
- 중위순회 (inorder traversal) : LVR
  - 왼쪽 자손 노드, 현재 노드, 오른쪽 자손 노드 순으로 방문한다.
- 후위순회 (postorder traversal) : LRV
  - 현재 노드보다 자손 노드를 먼저 방문한다.



<그림20_ SW문제해결 응용 _ 백트래킹_ 이진 트리의 순회>

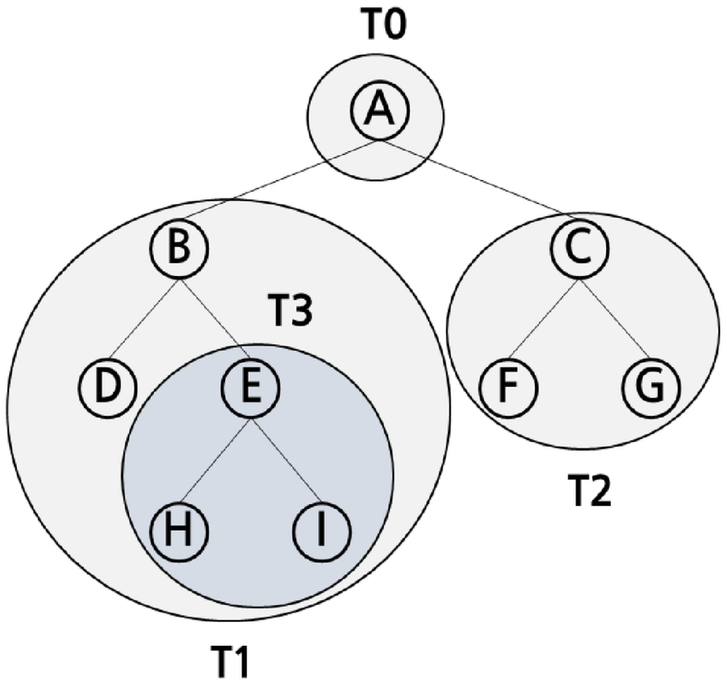
전위 순회 (preorder traversal)

- **수행 방법**
  ① 현재 노드 n을 방문하여 처리한다. : V
  ② 현재 노드 n의 왼쪽 서브트리를 순회한다. : L
  ③ 현재 노드 n의 오른쪽 서브트리를 순회한다. : R

- **전위 순회 알고리즘**

```
preorder_traverse (TREE T)
    IF T is not null
        visit(T)
        preorder_traverse(T.left)
        preorder_traverse(T.right)
```

전위 순회의 예



<그림21_ SW문제해결 응용 _ 백트래킹_ 전위 순회>

순서1 : T0 → T1 → T2

순서2 : A → B D (T3) → C F G

총 순서 : A B D E H I C F G

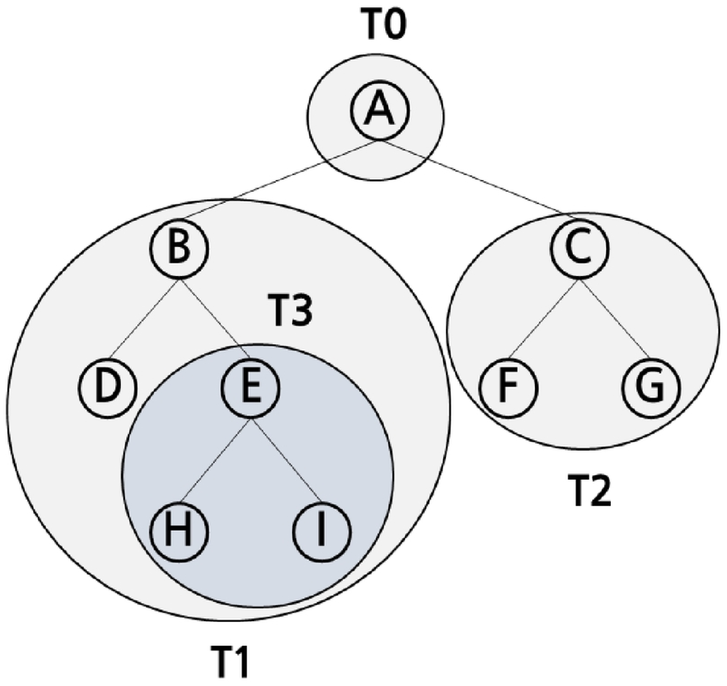
중위 순회 (inorder traversal)

- **수행 방법**
  ① 현재 노드 n의 왼쪽 서브트리를 순회한다. : L
  ② 현재 노드 n을 방문하여 처리한다. : V
  ③ 현재 노드 n의 오른쪽 서브트리로 순회한다. : R

- **중위 순회 알고리즘**

```
inorder_traverse (TREE T)
    IF T is not null
        inorder_traverse(T.left)
        visit(T)
        inorder_traverse(T.right)
```

중위 순회의 예



<그림22_ SW문제해결 응용 _ 백트래킹_ 중위 순회>

순서1 : T1 → T0 → T2

순서2 : D B (T3) → A → F C G

총 순서 : D B H E I A F C G

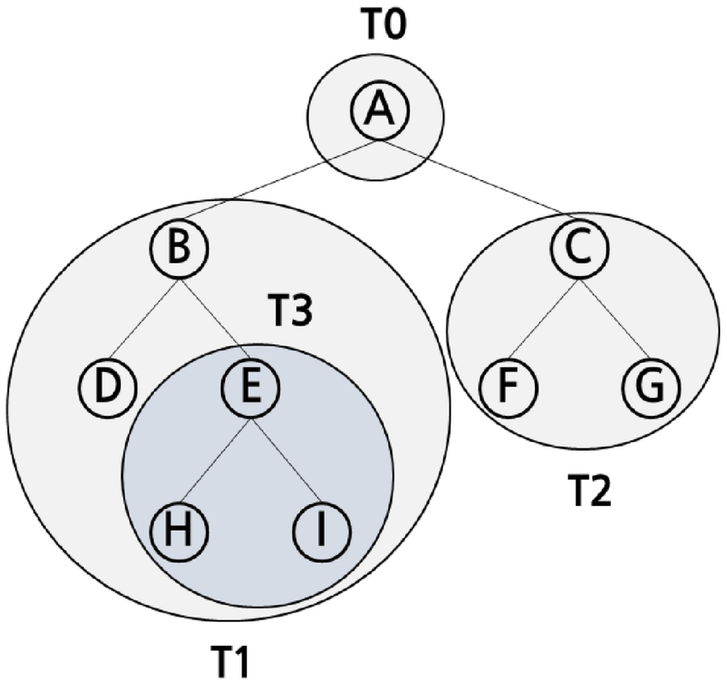
후위 순회 (postorder traversal)

- **수행 방법**
  ① 현재 노드 n의 왼쪽 서브트리를 순회한다. : L
  ② 현재 노드 n을 오른쪽 서브트리를 순회한다. : R
  ③ 현재 노드 n을 방문하여 처리한다. : V

- **후위 순회 알고리즘**

```
postorder_traverse(TREE T)
    IF T is not null
        postorder_traverse(T.left)
        postorder_traverse(T.right)
        visit(T)
```

후위 순회의 예



<그림23_ SW문제해결 응용 _ 백트래킹_ 후위 순회>

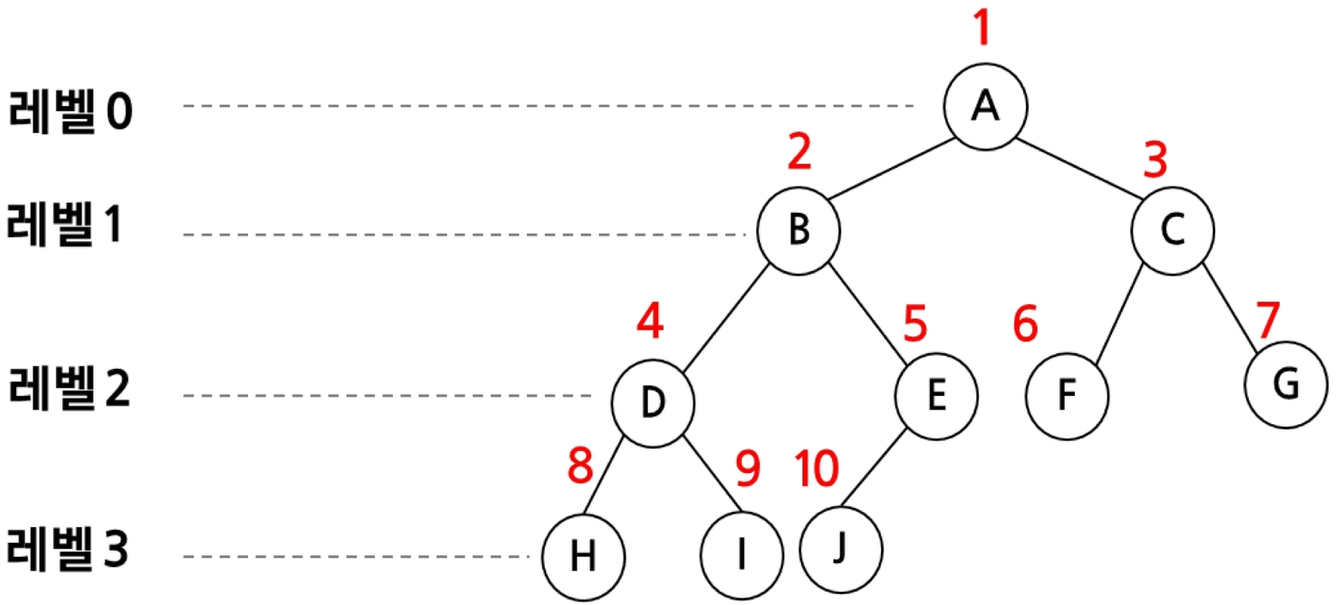
순서1 : T1 → T2 → T0

순서2 : D (T3) B → F G C → A

총 순서 : D H I E B F G C A

배열을 이용한 이진 트리의 표현

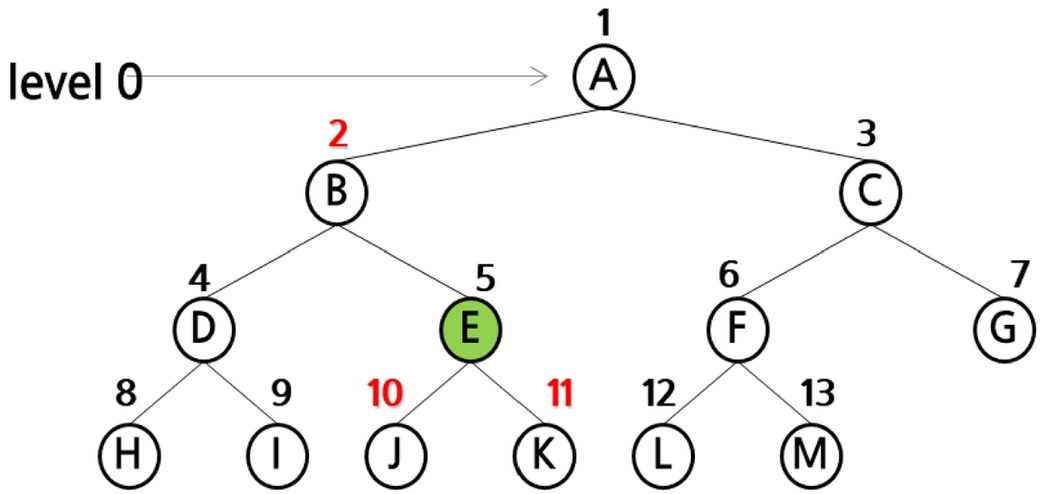
- 이진 트리에 각 노드 번호를 다음과 같이 부여
- 루트의 번호를 1로 부여하고
- 레벨 n에 있는 노드에 대하여 왼쪽부터 오른쪽으로 2^n 부터 $2^{n+1}-1$ 까지 번호를 차례로 부여



<그림24_ SW문제해결 응용 _ 백트래킹_ 노드 번호가 부여된 이진 트리>

노드 번호의 성질

- 노드 번호가 i인 노드의 부모 노드 번호? i//2
- 노드 번호가 i인 노드의 왼쪽 자식 노드 번호? 2*i
- 노드 번호가 i인 노드의 오른쪽 자식 노드 번호? 2*i + 1
- 레벨 n의 노드 번호 시작 번호는? 2^n

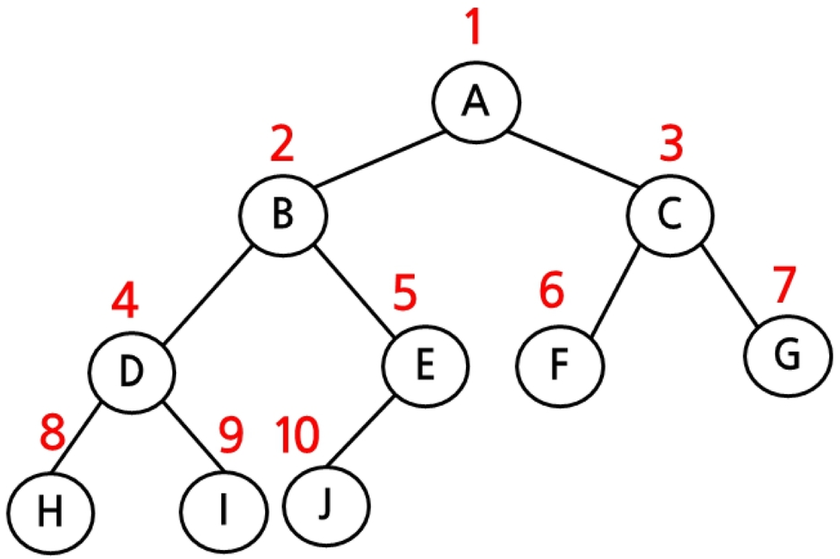


<그림25_ SW문제해결 응용 _ 백트래킹_ 노드 번호의 성질>

<그림26_ SW문제해결 응용 _ 백트래킹_ 노드 번호 찾기>

배열을 이용한 이진 트리의 표현

- 노드 번호를 배열의 인덱스로 사용
- 높이가 h인 이진 트리를 위한 배열의 크기는?
  - 레벨 i의 최대 노드 수는? 2^i
  - 따라서 $1 + 2 + 4 + 8 \cdots + 2^i = 2^{h+1}-1$

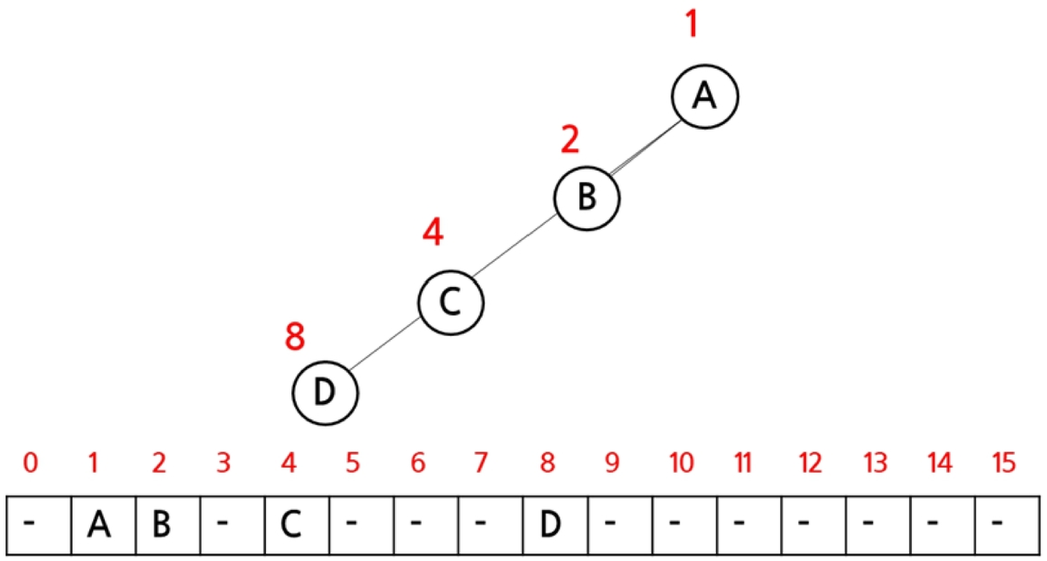


<그림28_ SW문제해결 응용 _ 백트래킹_ 배열을 이용한 이진 트리>

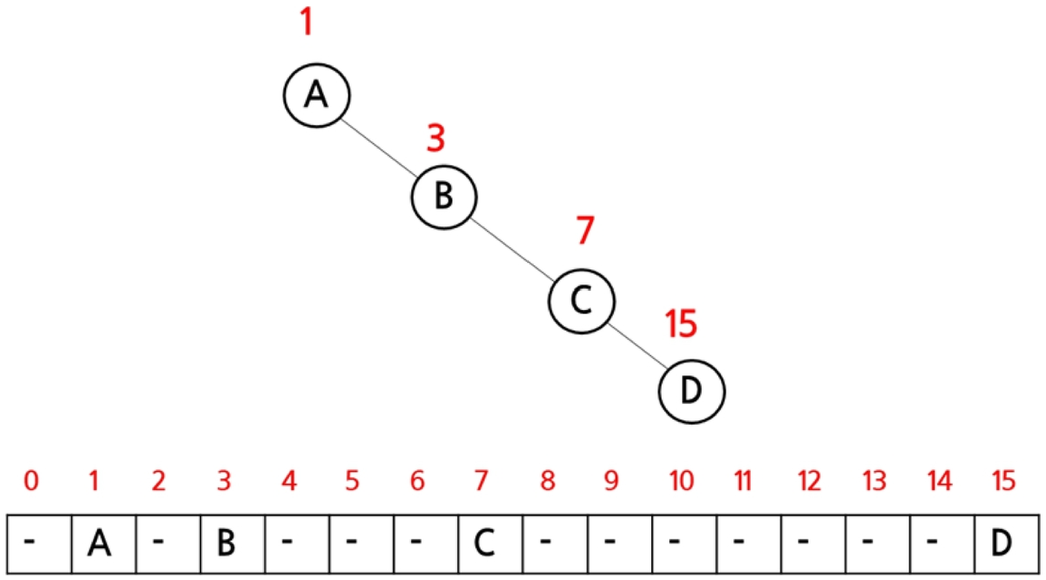
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-	A	B	C	D	E	F	G	H	I	J	-	-	-	-	-

<그림27_ SW문제해결 응용 _ 백트래킹_ 노드 번호를 배열의 인덱스로 사용하기>

배열을 이용한 이진 트리의 표현



<그림29_ SW문제해결 응용 _ 백트래킹_ 배열을 이용한 왼쪽 편향 이진 트리>



<그림30_ SW문제해결 응용 _ 백트래킹_ 배열을 이용한 오른쪽 편향 이진 트리>

배열을 이용한 이진 트리의 표현의 단점

- 편향 이진 트리의 경우에 사용하지 않는 배열 원소에 대한 메모리 공간 낭비 발생
- 트리의 중간에 새로운 노드를 삽입하거나 기존의 노드를 삭제할 경우 배열의 크기 변경 어려워 비효율적

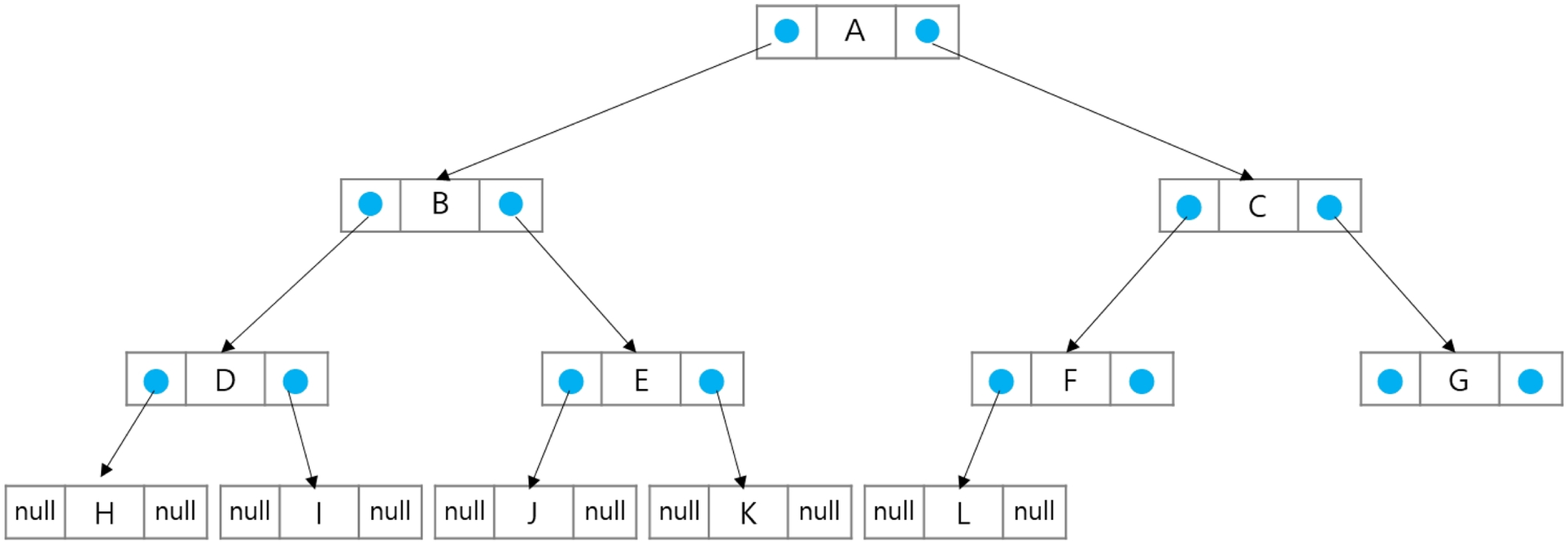
※ 위 단점을 보완하기 위해 연결리스트를 이용하여 트리를 표현할 수 있다.

연결리스트를 이용한 이진 트리의 표현

- 이진 트리의 모든 노드는 최대 2개의 자식 노드를 가지므로 일정한 구조의 단순 연결 리스트 노드를 사용하여 구현

<그림31_ SW문제해결 응용 _ 백트래킹_ 연결리스트를 이용한 이진 트리의 표현>

완전 이진 트리의 연결 리스트 표현



<그림32_ SW문제해결 응용 _ 백트래킹_ 완전 이진 트리의 연결 리스트 표현>

연습 문제

이진 트리를 전/중/후위 순회하고 방문한 노드의 번호를 출력

- 첫 줄에는 트리의 노드의 총 수 V가 주어진다.
- 그 다음 줄에는 V-1개 간선이 나열된다. 간선은 그것을 이루는 두 정점으로 표기된다. 간선은 항상 “부모 자식” 순서로 표기된다.
- 아래 예에서 두 번째 줄 처음 1과 2는 정점 1과 2를 잇는 간선을 의미하며 1이 부모, 2가 자식을 의미한다..

12

1 2 1 3 2 4 3 5 3 6 4 7 5 8 5 9 6 10 6 11 7 12 11 13

- 코드 : 연습문제3_트리순회.py

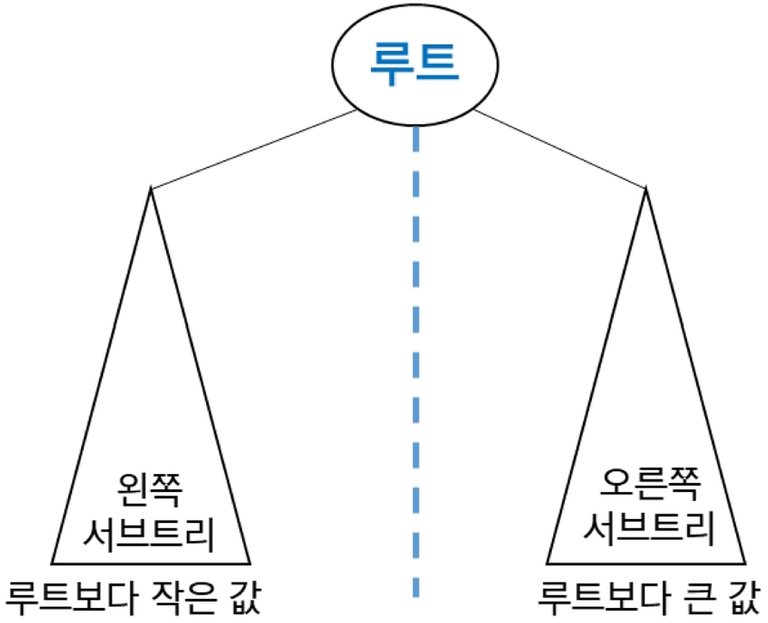
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

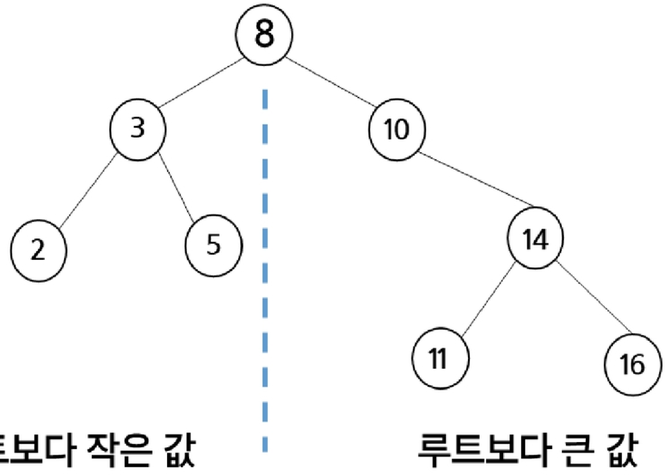
이진탐색트리

이진탐색트리 (BST, Binary Search Tree)

- 탐색 작업을 효율적으로 하기 위한 자료구조
- 모든 원소는 서로 다른 유일한 키를 가짐
- key (왼쪽 서브트리) < key (루트 노드) < key (오른쪽 서브트리)
- 왼쪽 서브트리와 오른쪽 서브트리도 이진탐색트리
- 중위 순회하면 오름차순으로 정렬된 값을 얻을 수 있음



<그림33_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리>



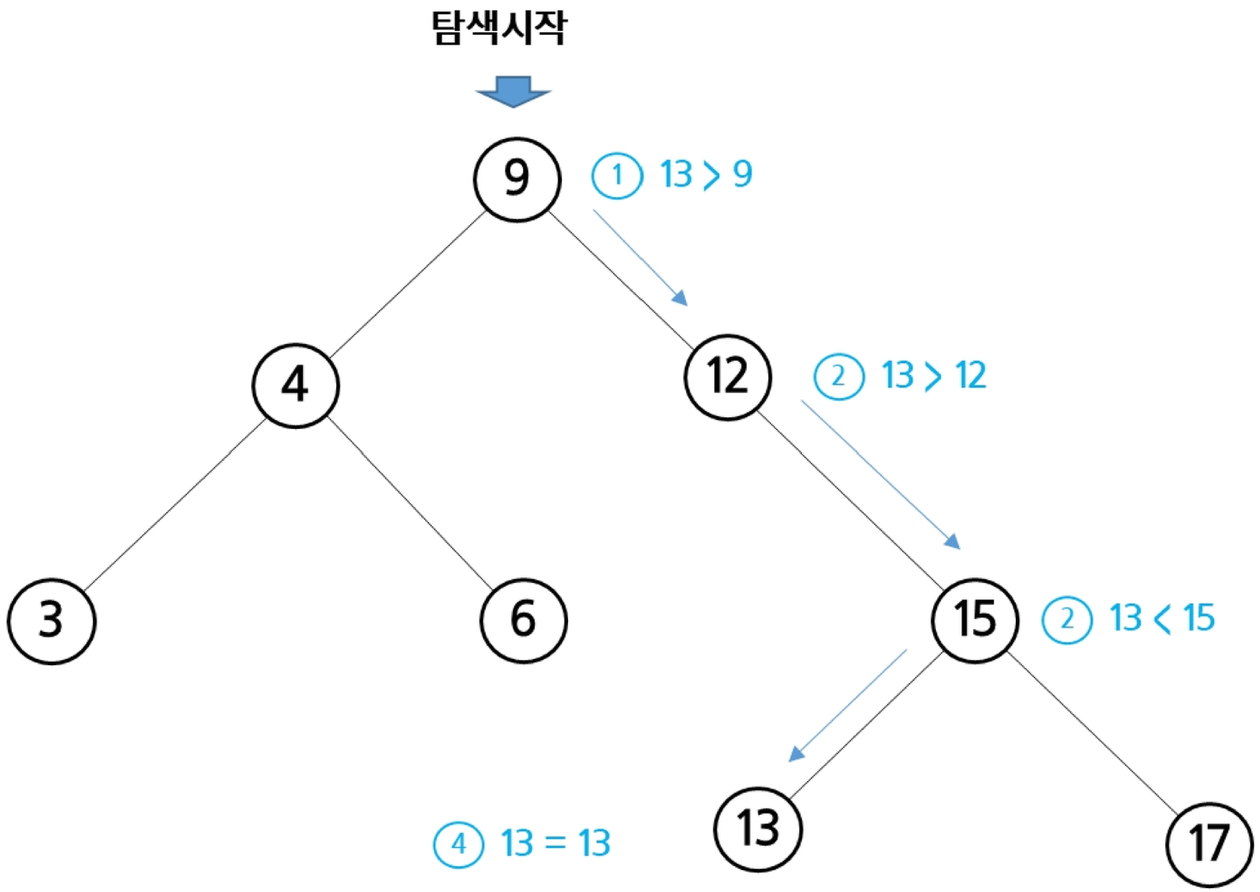
<그림34_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 예>

탐색 연산

- 루트에서 탐색 시작
- 탐색할 키 값 x를 루트 노드의 키 값 k와 비교
  - x == k : 탐색연산을 성공한 경우입니다.
  - x < k : 루트 노드의 왼쪽 서브트리에 대해서 탐색연산을 수행합니다.
  - x > k : 루트 노드의 오른쪽 서브트리에 대해서 탐색연산을 수행합니다.
- 서브트리에 대해서 순환적으로 탐색 연산을 반복
- 탐색 수행할 서브 트리가 없으면 탐색 실패

탐색 연산

- 13 탐색



<그림35_ SW문제해결 응용 _ 백트래킹_ 13 탐색 과정>

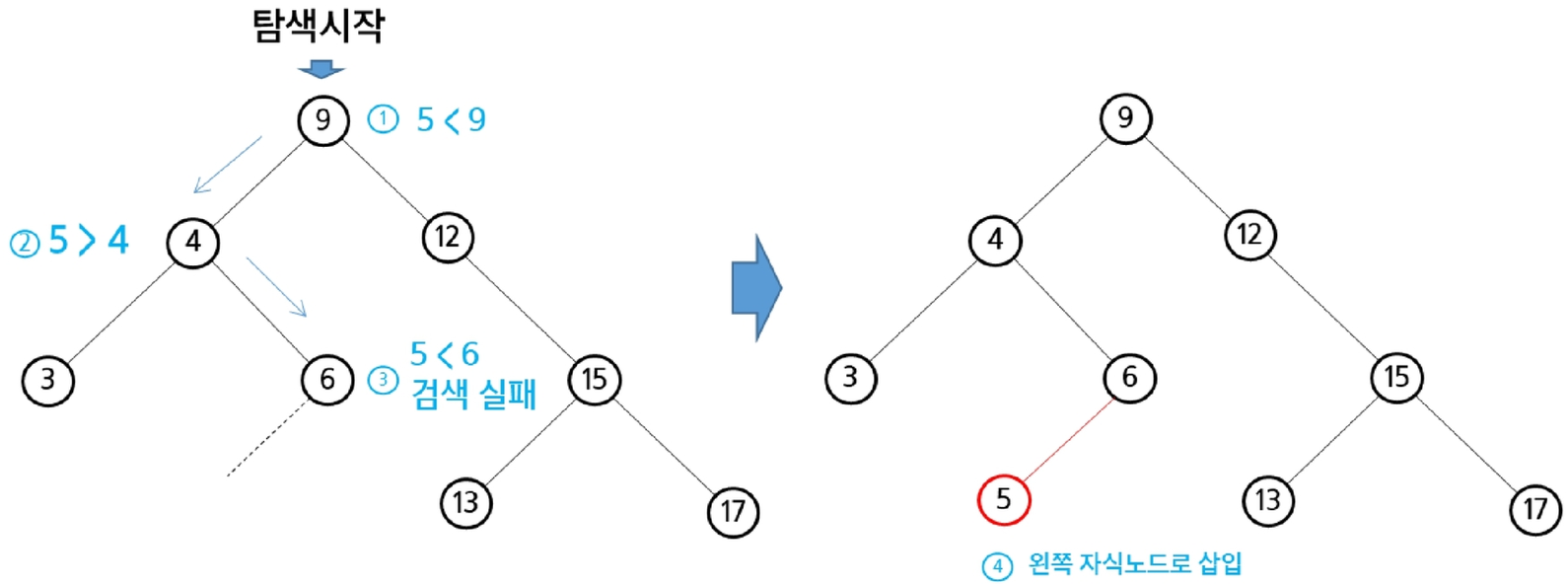
삽입 연산

① 먼저 탐색 연산을 수행

- 삽입할 원소와 같은 원소가 트리에 있으면 삽입할 수 없으므로, 같은 원소가 트리에 있는지 탐색하여 확인한다.
- 탐색에서 탐색 실패가 결정되는 위치가 삽입 위치가 된다.

② 탐색 실패한 위치에 원소를 삽입한다.

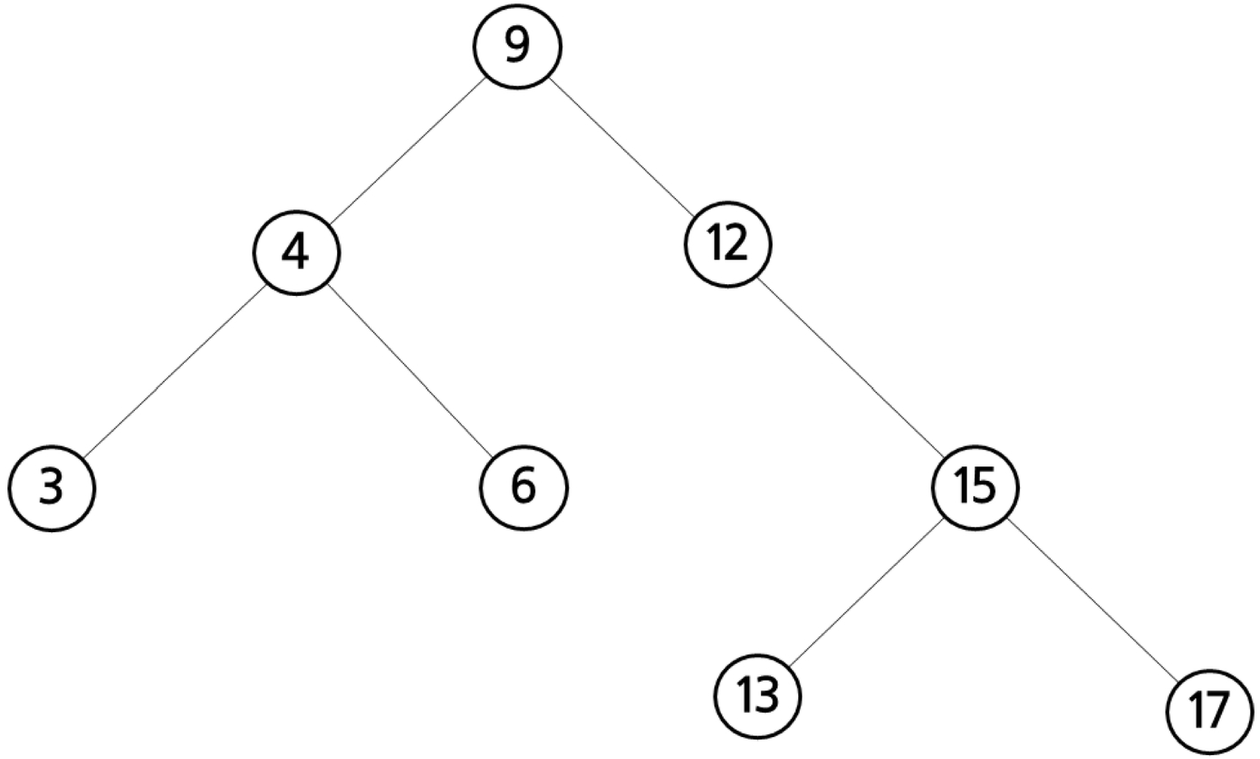
- 다음 예는 5를 삽입하는 예이다.



<그림36_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 삽입연산>

삭제 연산

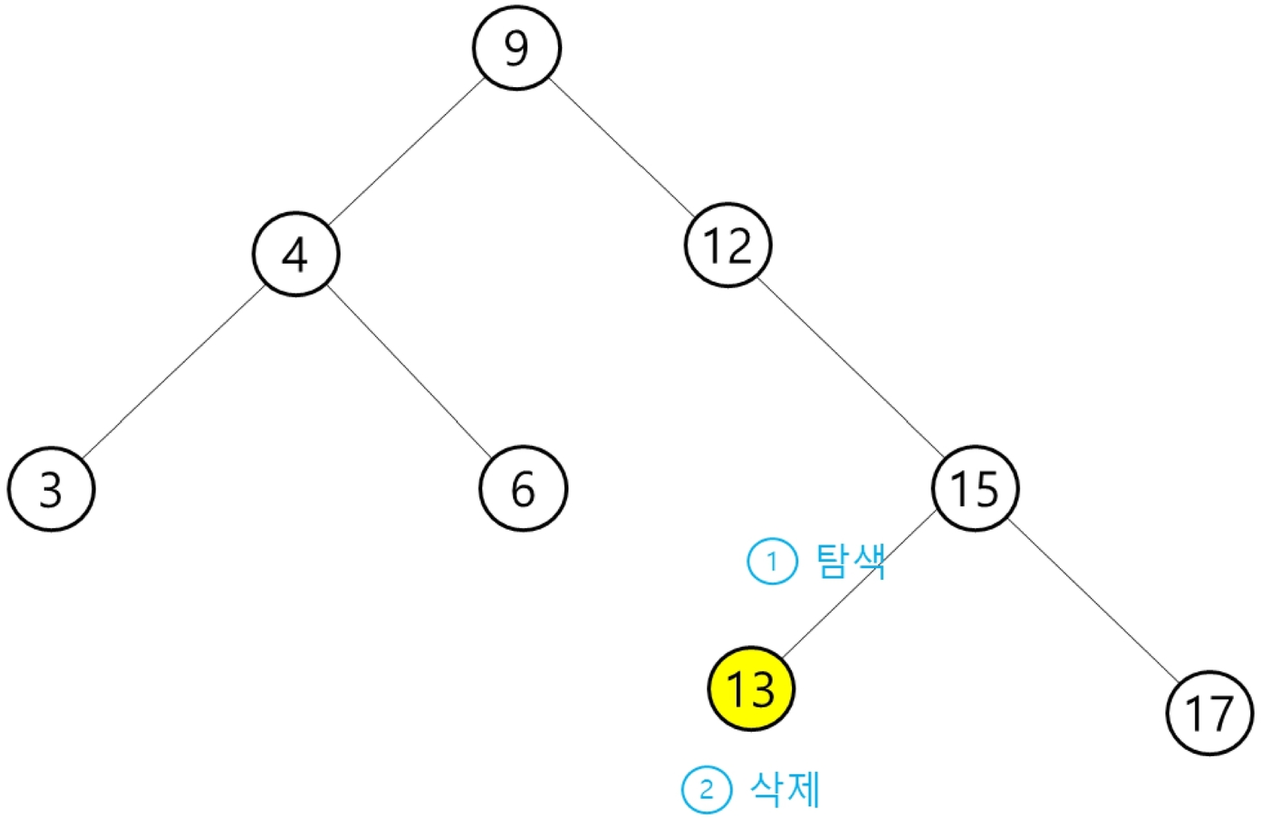
- 삭제 연산에 대해 알고리즘을 생각해 봅시다.
- 다음 트리에 대하여 13, 12, 9를 차례로 삭제해 보자.



<그림37_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 삭제연산>

13 삭제

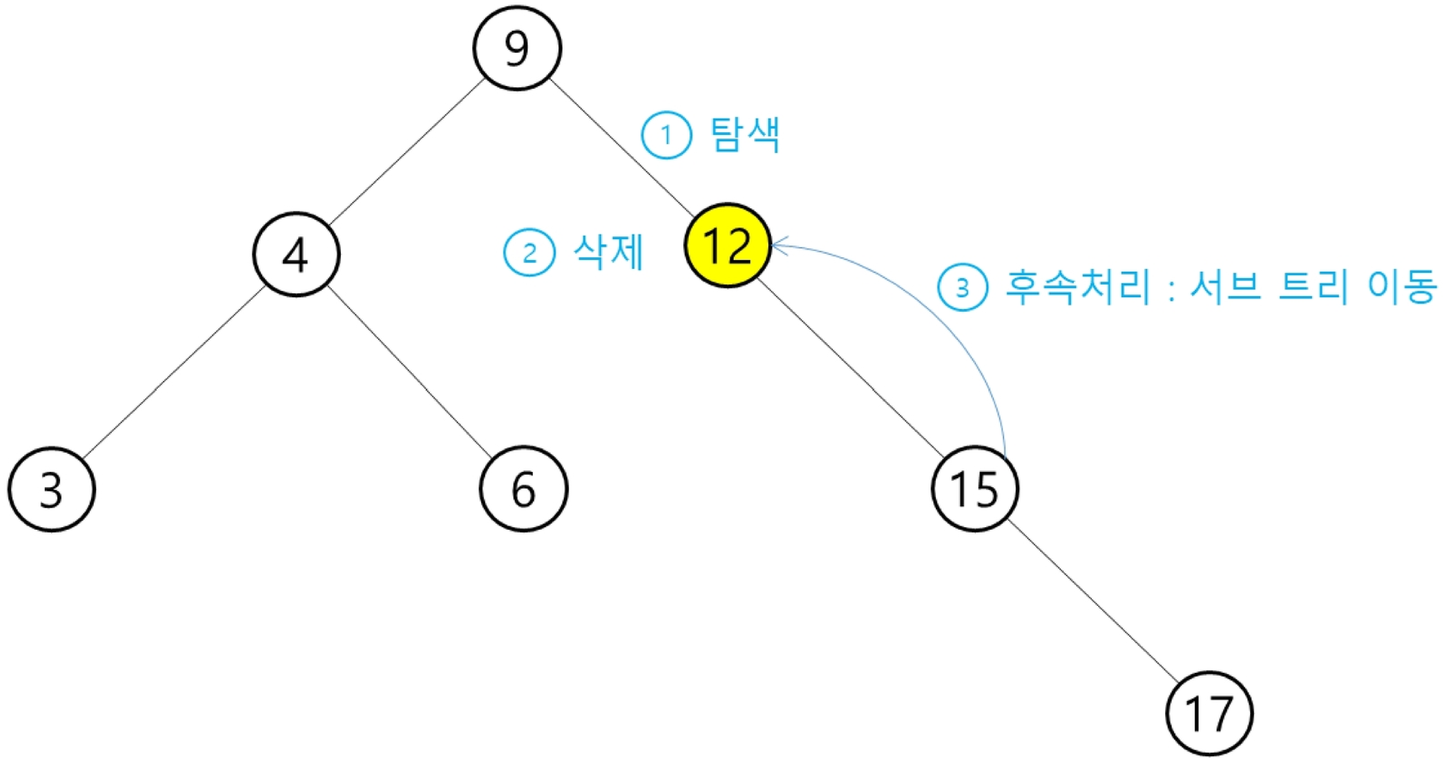
- 삭제할 노드가 리프 노드인 경우 : 차수가 0인 경우



<그림38_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 13 삭제 연산>

12 삭제

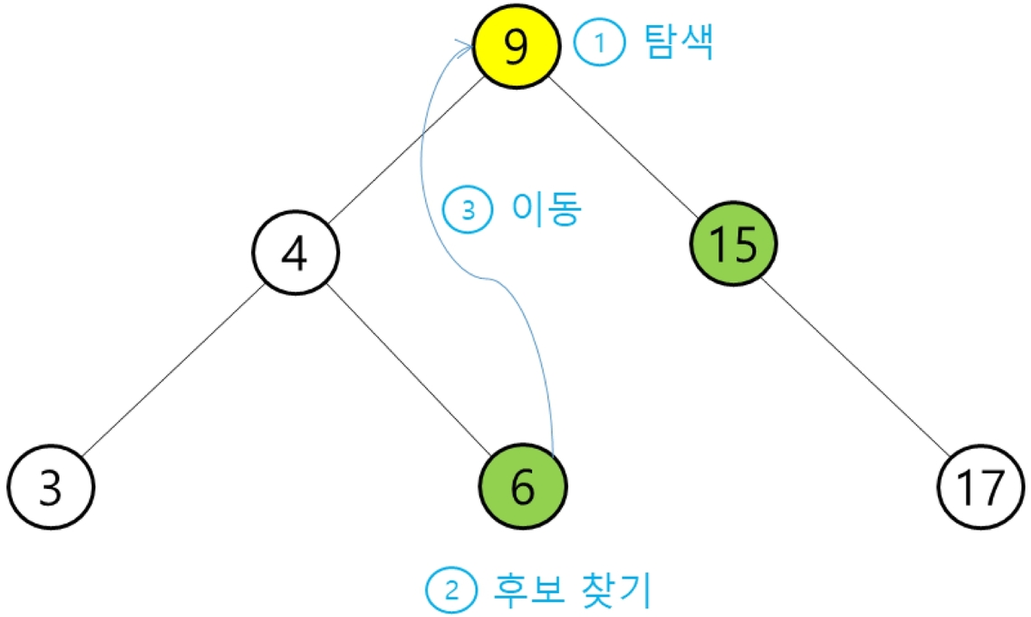
- 삭제할 노드가 리프 노드가 아닌 경우 : 차수가 1인 경우



<그림39_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 12 삭제 연산>

9 삭제

- 삭제할 노드가 리프 노드가 아닌 경우 : 차수가 2인 경우



<그림40_ SW문제해결 응용 _ 백트래킹_ 이진탐색트리의 9 삭제 연산>

이진탐색트리의 성능

- 탐색(searching), 삽입(insertion), 삭제(deletion) 시간은 트리의 높이만큼 시간이 걸린다.
  - O(h), h : BST의 깊이 (height)
- 평균의 경우
  - 이진 트리가 균형적으로 생성되어 있는 경우
  - O(log n)
- 최악의 경우
  - 한쪽으로 치우친 경사 이진트리의 경우
  - O(n)
  - 순차탐색과 시간 복잡도가 같다.

SAMSUNG SW AI ACADEMY FOR YOUTH

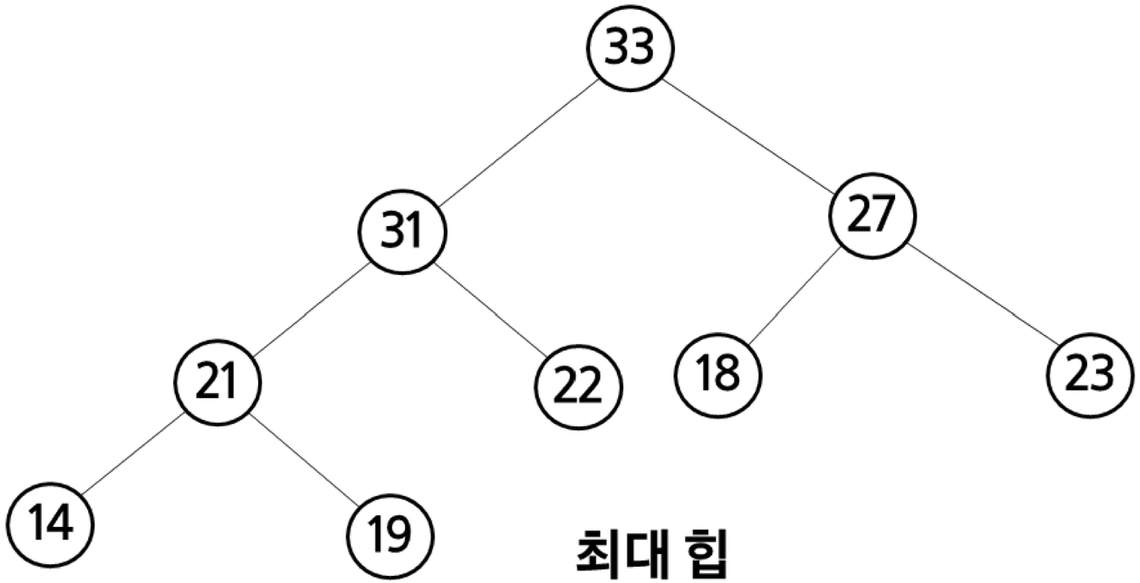
함께가요 미래로!
Enabling People

힙

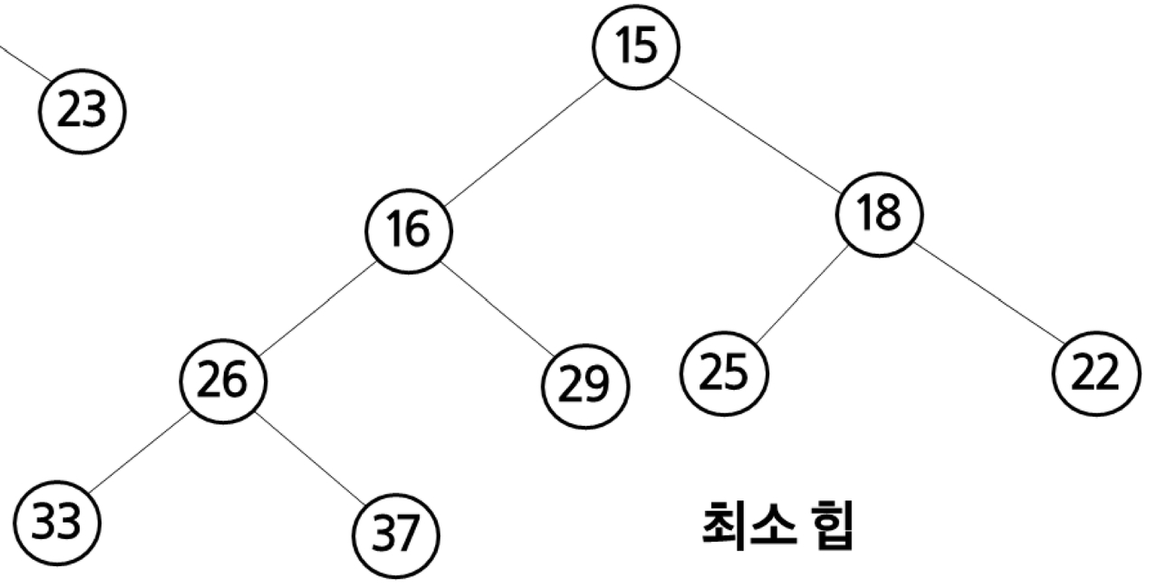
힙 (heap)

- 완전 이진 트리에 있는 노드 중에서 키 값이 가장 큰 노드나 키 값이 가장 작은 노드를 찾기 위해서 만든 자료 구조
- 최대 힙 (max heap)
  - 키 값이 가장 큰 노드를 찾기 위한 힙 입니다.
  - 부모 노드의 키 값 > 자식 노드의 키 값
  - 루트 노드는 키 값이 가장 큰 노드입니다.
- 최소 힙 (min heap)
  - 키 값이 가장 작은 노드를 찾기 위한 힙 입니다.
  - 부모 노드의 키 값 < 자식 노드의 키 값
  - 루트 노드는 키 값이 가장 작은 노드입니다.

힙의 예



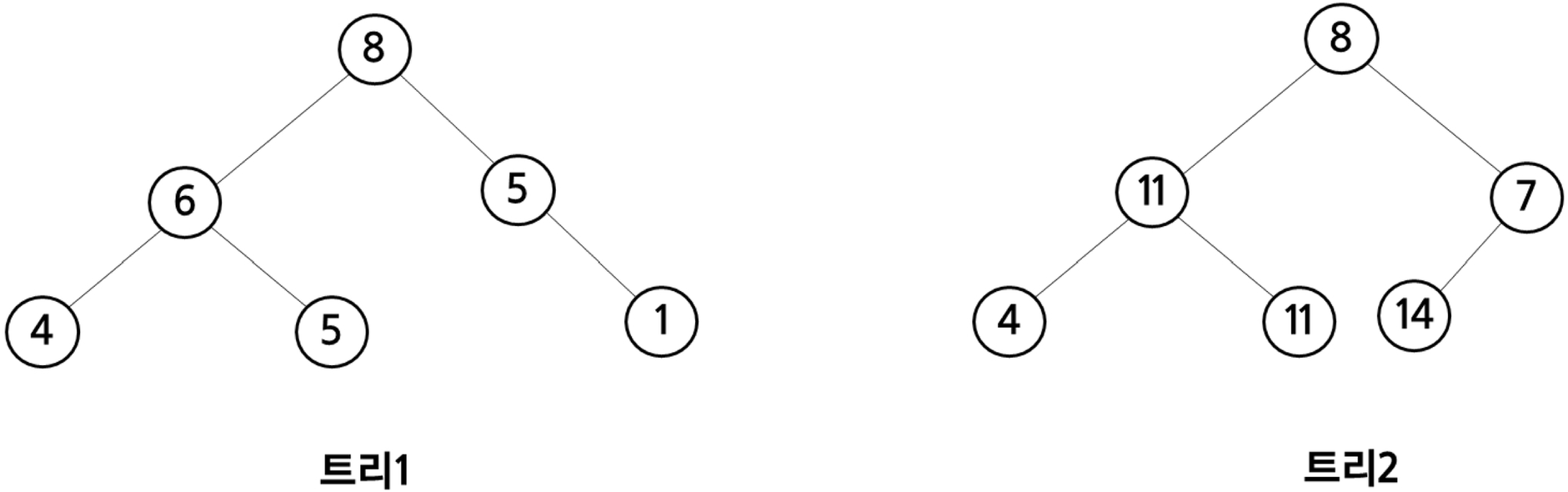
<그림41_ SW문제해결 응용 _ 백트래킹_ 최대 힙>



<그림42_ SW문제해결 응용 _ 백트래킹_ 최소 힙>

힙이 아닌 이진 트리의 예

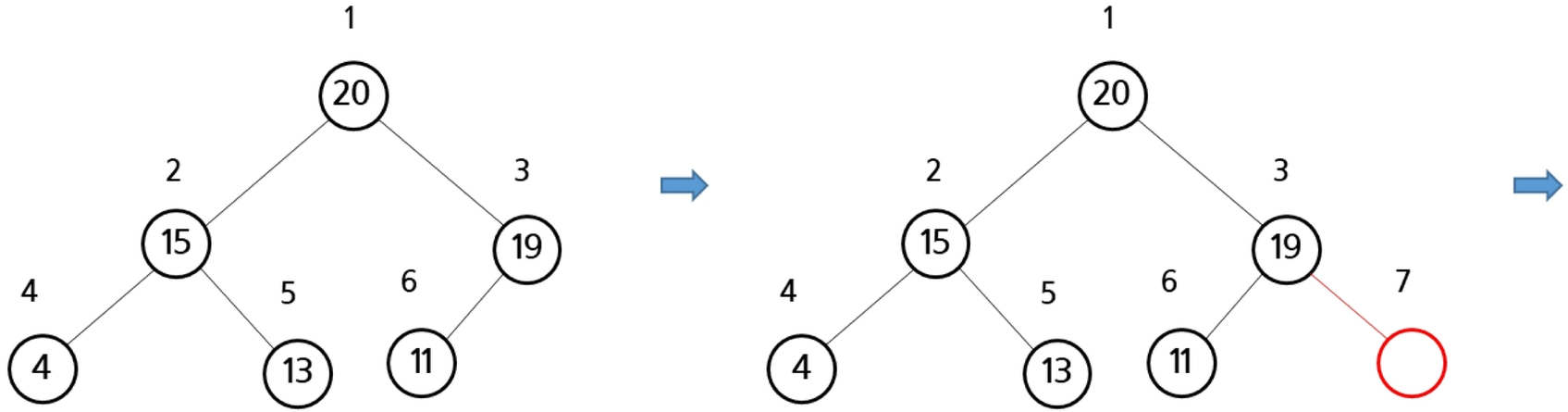
- 아닌 이유를 설명해 보세요



<그림43_ SW문제해결 응용 _ 백트래킹_ 힙이 아닌 이진 트리의 예>

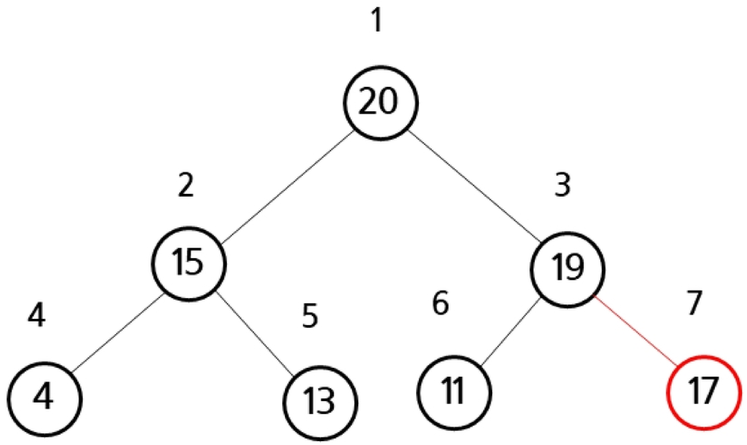
힙 연산 - 삽입

- 17 삽입



삽입 전의 힙

삽입할 자리 확장

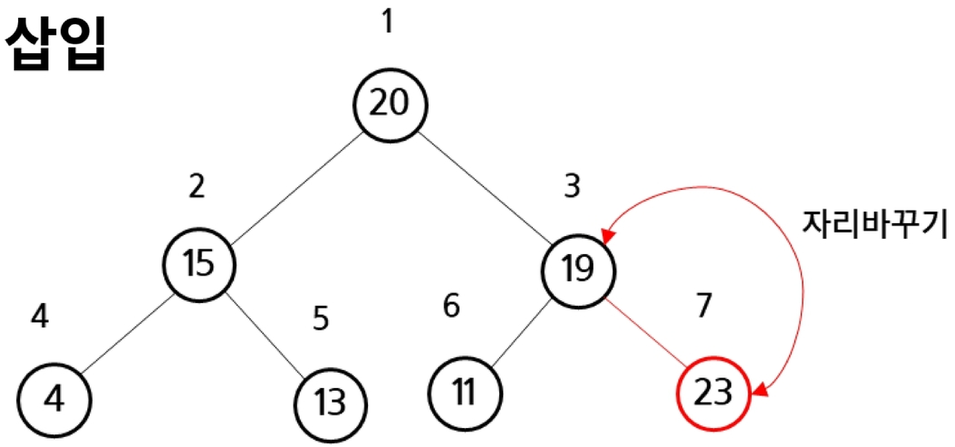


확장한 자리에 삽입할 원소 저장

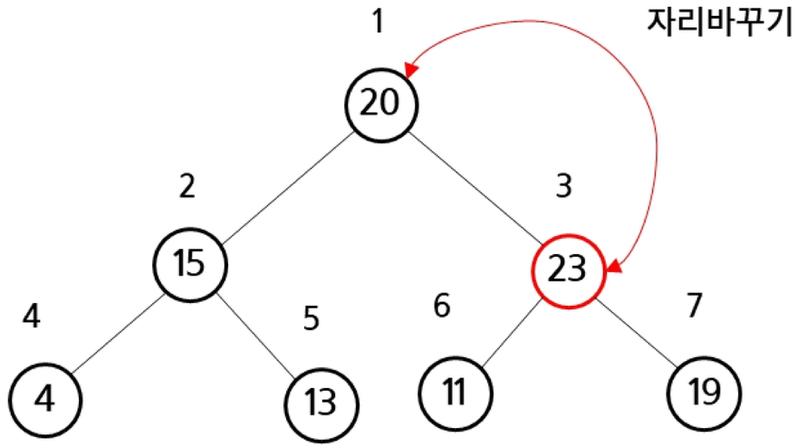
<그림44_ SW문제해결 응용 _ 백트래킹_ 힙의 17 삽입 연산>

힙 연산 - 삽입

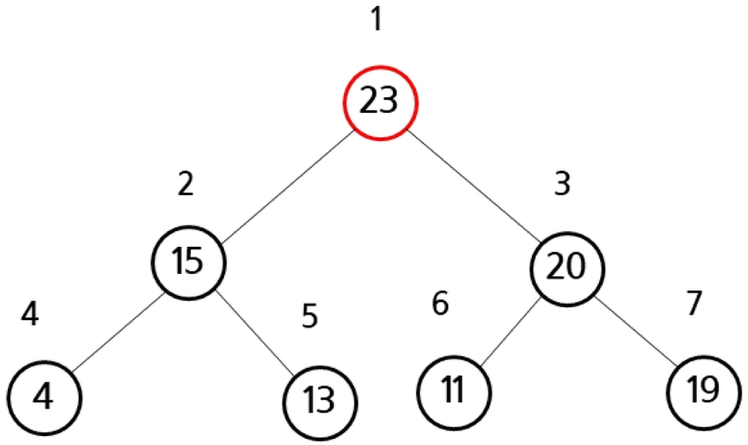
- 23 삽입



1) (삽입 노드 23 > 부모 노드 19) : 자리바꾸기



2) (삽입 노드 23 > 부모 노드 19) : 자리바꾸기



3) 비교할 부모 노드가 없으므로 자리 확정

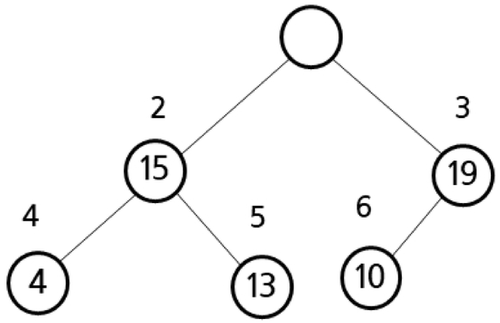
<그림45_ SW문제해결 응용 _ 백트래킹_ 힙의 23 삽입 연산>

힙 연산 - 삭제

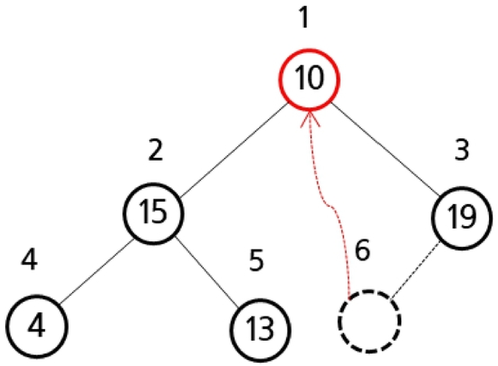
- 힙에서는 루트 노드의 원소만을 삭제할 수 있다.
- 루트 노드의 원소를 삭제하여 반환한다.
- 힙의 종류에 따라 최대값 또는 최소값을 구할 수 있다.
  - 우선순위 큐와 비교

힙 연산 - 삭제

루트의 원소 삭제

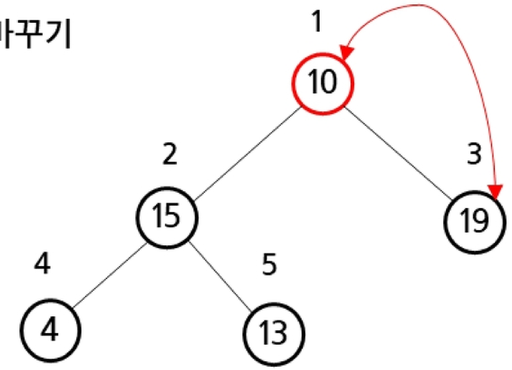


1) 루트 노드의 원소 삭제

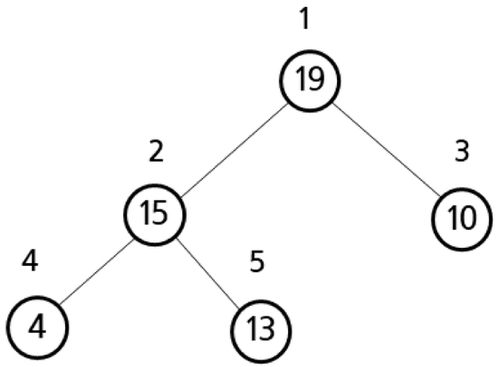


2) 마지막 노드 삭제

자리 바꾸기



3) (삽입노드 10 < 자식노드 19) : 자리바꾸기



4) 자리 확정

<그림45_ SW문제해결 응용 _ 백트래킹_ 힙의 삭제 연산>

힙의 활용

- 힙을 활용하는 대표적인 2가지 예는 우선순위 큐의 구현과 정렬
- 우선순위 큐를 구현하는 가장 효율적인 방법이 힙을 사용하는 것
  - 노드 하나의 추가/삭제가 시간 복잡도가 O(log n)이고 최대값/최소값을 O(1)에 구할 수 있습니다.
- 힙 정렬은 O(n log n)을 보장
- 코드 : 3.heap.py

힙의 활용

- 배열을 통해 트리 형태를 쉽게 구현 가능
  - 부모나 자식 노드를 O(1) 연산으로 쉽게 찾을 수 있습니다.
  - n번 위치에 있는 노드의 자식은 2n 과 2n+1 번에 위치합니다.
  - 완전 이진 트리의 특성에 의해 추가/삭제의 위치는 자료의 시작과 끝 인덱스로 쉽게 판단할 수 있습니다.

힙의 활용

- 힙 정렬은 힙 자료구조를 이용해서 이진 탐색과 유사한 방법으로 수행됨
- 정렬을 위한 2단계
  ① N개의 노드 삽입 연산 + N개의 노드 삭제 연산
  ② 힙에서 순차적 (오름차순)으로 값을 하나씩 제거
- 힙 정렬의 시간 복잡도
  - N개의 노드 삽입 연산 + N개의 노드 삭제 연산이 필요합니다.
  - 삽입과 삭제 연산은 각각 O(logN)입니다.
  - 따라서, 전체 정렬은 O(NlogN)입니다.
- 힙 정렬은 배열에 저장된 자료를 정렬하기에 유용함

실습

백트래킹

- 5208. 전기버스2
- 5209. 최소 생산 비용

이진탐색트리

- 5176. 이진탐색

힙

- 5177. 이진 힙

추가연습

- 2806. N-Queen

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 백트래킹 알고리즘의 핵심 개념은?

a) 이전 값을 저장해두고 다시 사용한다.
b) 조건을 만족하지 않으면 이전 단계로 되돌아간다.
c) 모든 경우를 무작위로 선택한다.
d) 탐색 전에 항상 정렬을 수행한다.

2 다음 중 대표적인 백트래킹 문제로 알려진 것은?

a) 평균값 계산
b) 버블 정렬
c) N-Queen 문제
d) 최소값 찾기

3 상태 공간 트리에서 노드는 무엇을 나타내는가?

a) 전체 입력값
b) 선택 가능한 모든 원소
c) 현재까지의 선택 상태
d) 모든 결과의 평균

4 다음 중 가지치기에 해당하는 설명은?

a) 모든 경우를 끝까지 탐색
b) 이미 선택한 값을 다시 사용하는 것
c) 유망하지 않은 경로는 탐색하지 않음
d) 모든 경로를 반복문으로 처리

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 순열을 구할 때 적절한 백트래킹 조건은?

a) 모든 원소 선택 가능
b) 이미 선택한 원소는 다시 선택하지 않음
c) 항상 오름차순으로 선택
d) 같은 수를 여러 번 선택

6 조합과 순열의 차이는?

a) 조합은 순서를 고려하지 않는다.
b) 순열은 일부만 선택한다.
c) 조합은 중복을 허용한다.
d) 순열은 항상 2개씩 짝짓는다.

7 다음 중 상태 공간 트리로 표현하기 적절한 것은?

a) 파일 저장 경로
b) 퀸의 위치를 한 줄씩 정하는 방법
c) 정렬된 수열
d) 입력값의 평균값

8 백트래킹에서 재귀함수의 역할은?

a) 모든 경우를 병렬로 실행
b) 조건문 없이 자동 종료
c) 이전 상태로 돌아가도록 호출을 반복
d) 반복문 없이 모든 연산 수행

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 중 백트래킹 탐색 중단 조건으로 가장 적절한 것은?

a) 아직 선택하지 않은 값이 남았을 때
b) 현재 경로가 문제의 조건을 만족하지 않았을 때
c) 재귀 함수가 처음 호출되었을 때
d) 반복문이 종료되지 않았을 때

10 다음 중 조합(Combination)에 해당하는 예시는?

a) [1, 2], [2, 1]
b) [3, 4], [4, 3]
c) [1, 2], [1, 3]
d) [1, 1], [2, 2]

11 백트래킹을 구현할 때 함수의 종료 조건은 어떤 역할을 하는가?

a) 조건 없이 무한 반복되게 한다.
b) 방문한 노드를 모두 다시 방문하게 한다.
c) 하나의 가능한 해를 완성했을 때 탐색을 멈춘다.
d) 항상 루트 노드에서만 실행된다.

12 다음 중 상태 공간 트리를 사용하는 가장 큰 이유는 무엇인가?

a) 입력값을 정렬하기 위해
b) 탐색 순서를 무작위로 바꾸기 위해
c) 가능한 모든 선택 경로를 구조화하여 탐색하기 위해
d) 데이터를 이진 탐색하기 위해

확인문제

정답 및 해설

1. b) 조건을 만족하지 않으면 이전 단계로 되돌아간다. 2. c) N-Queen 문제 3. c) 현재까지의 선택 상태 4. c) 유망하지 않은 경로는 탐색하지 않음 5. b) 이미 선택한 원소는 다시 선택하지 않음 6. a) 조합은 순서를 고려하지 않는다. 7. b) 퀸의 위치를 한 줄씩 정하는 방법
8. c) 이전 상태로 돌아가도록 호출을 반복 9. b) 현재 경로가 문제의 조건을 만족하지 않았을 때 10. c) [1, 2], [1, 3] 11. c) 하나의 가능한 해를 완성했을 때 탐색을 멈춘다. 12. c) 가능한 모든 선택 경로를 구조화하여 탐색하기 위해

1. 백트래킹은 조건을 만족하지 않으면 이전 단계로 돌아가며 다른 경우를 시도합니다.

2. N-Queen 문제는 조건에 따라 탐색 경로를 제한하는 백트래킹이 적합합니다.

3. 상태공간트리에서 각 노드는 현재까지의 선택 상태를 표현합니다.

4. 가지치기는 조건을 만족하지 않는 경로를 미리 제외하는 전략입니다.

5. 순열은 순서를 고려하므로 이미 선택한 원소는 중복 사용하지 않습니다.

6. 조합은 순서를 고려하지 않으며, 순열은 순서를 고려합니다.

7. 퀸의 위치를 각 행마다 정하는 방식은 상태공간트리로 표현 가능합니다.

8. 재귀를 통해 현재 상태에서 다음 상태로 이동하고, 실패 시 다시 돌아갈 수 있습니다.

9. 조건을 만족하지 않으면 해당 경로의 탐색을 중단해야 합니다. (가지치기)

10. 조합은 순서를 고려하지 않고 일부 원소를 선택하는 것이므로 [1, 2], [1, 3]은 조합입니다.

11. 재귀 함수에서 종료 조건은 해를 하나 완성한 경우 탐색을 마무리하게 해 줍니다.

12. 상태공간트리는 문제 해결을 위한 가능한 모든 경로를 트리 구조로 표현하여 체계적으로 탐색할 수 있게 합니다.

활동정리

핵심키워드

개념	설명	예시
백트래킹	모든 경우를 시도하면서 조건을 만족하지 않으면 되돌아가는 탐색 방식	N-Queen, 부분집합, 순열 문제 해결
상태공간트리	가능한 해들을 트리 형태로 표현하며, 노드마다 상태를 나타냄	퀸을 각 행에 놓는 모든 배치 표현
가지치기	정답이 될 수 없는 경우의 분기는 미리 제거함	대각선에 퀸이 겹치면 다음 탐색 생략
방문 처리	중복 탐색을 방지하기 위해 현재 경로에서 사용된 원소를 표시	visited[i] = True
순열	순서를 고려한 원소들의 나열을 모두 구함	[1, 2, 3] → [1, 2, 3], [2, 1, 3], …
조합	순서를 고려하지 않고 일부 원소를 선택함	[1, 2, 3] 중 2개 선택 : [1, 2], [2, 3], …
트리 순회	트리 구조에서 노드를 일정 순서대로 방문함	전위 순회, 중위 순회, 후위 순회

〈표1_ SW문제해결 응용 _ 백트래킹_ 핵심키워드〉

활동정리

오늘 공부한 내용 요약 및 정리

- **백트래킹이란?**
  - 가능한 모든 경우를 탐색하면서 조건을 만족하지 않으면 되돌아가는 알고리즘
  - 불필요한 탐색을 줄이기 위해 “가지치기” 기법 사용

- **상태 공간 트리의 구성**
  - 문제 해결 과정을 트리 구조로 시각화
  - 한 단계씩 선택지를 확장하며 탐색
  - 노드는 현재 상태, 간선은 선택 흐름

활동정리

오늘 공부한 내용 요약 및 정리

- **트리 순회와 탐색 방식**
  - 전위, 중위, 후위순회 방식으로 노드 순서 결정
  - DFS(깊이 우선 탐색) 기반으로 탐색 흐름 이해
  - 순서에 따라 탐색 결과가 달라짐

- **순열과 조합**
  - 백트래킹은 순서가 있는 경우(순열), 일부를 선택하는 경우(조합)에 모두 사용됨
  - 문제에 따라 탐색 조건과 범위가 달라짐

활동정리

‘미로를 빠져나오는 법을 컴퓨터가 스스로 알아내게 할 수 있을까요?’

1. 여러 경로를 탐색하다가 조건에 맞지 않으면 다시 돌아가는 방식을 [ ]이라고 합니다.
2. 탐색 중에 가능성이 없는 경우를 미리 제외하는 과정을 [ ]라고 합니다.

활동정리

‘미로를 빠져나오는 법을 컴퓨터가 스스로 알아내게 할 수 있을까요?’

1. 여러 경로를 탐색하다가 조건에 맞지 않으면 다시 돌아가는 방식을 백트래킹이라고 합니다.
2. 탐색 중에 가능성이 없는 경우를 미리 제외하는 과정을 가지치기라고 합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다